DAC-203 | ARTIFICIAL INTELLIGENCE

Mehta Family School for Data Science and Artificial Intelligence

ADAPTIVE RESTART AND NOGOOD MANAGEMENT FOR CONSTRAINT SATISFACTION SOLVERS

A Quantitative Search-Control Architecture in CDCL SAT Solvers

ACADEMIC RESEARCH PROJECT REPORT

Submitted for DAC-203: Artificial Intelligence

Faculty Advisor: Prof. Manoj Kumar

Manav Garg | 25125026

Kugashiya Bhavy | 25125024

Implementation: Glucose 4.2.1, C++ (Windows 64-bit)

November 2026

Declaration of Originality

We declare that this report describes our research and engineering project, *Adaptive Restart and Nogood Management for Constraint Satisfaction Solvers*. It builds upon the open-source Glucose 4.2.1 SAT solver and published research, which are acknowledged and cited. The descriptions of algorithms, benchmark data and limitations are drawn from the project documentation and the supplied verification audit. We accept responsibility for reviewing the final text and approving any declarations before submission.

Manav Garg (25125026) — Signature: _____ Date: _____

Kugashiya Bhavy (25125024) — Signature: _____ Date: _____

Department: Mehta Family School for Data Science and Artificial Intelligence

Acknowledgements

We thank Prof. Manoj Kumar for academic supervision and acknowledge the contributions of the SAT-solving research community. In particular, the open-source Glucose solver by Gilles Audemard and Laurent Simon provided the technical foundation for this work. We also acknowledge the prior literature on rational metareasoning, variable and restart heuristics, constraint satisfaction, and clause learning, cited throughout this report.

Abstract

Modern conflict-driven clause learning (CDCL) SAT solvers depend on two tightly coupled mechanisms: restarts, which reset the current decision trail without discarding valid learned knowledge, and learned-clause database reduction, which limits the propagation cost of retaining too many clauses. This study implemented and evaluated a family of lightweight search-control methods in a native Windows port of Glucose 4.2.1. The investigated variants include propagation-efficiency restarts (V1/V2), a finance-inspired maximum-drawdown controller (V3), multi-attribute Adaptive-Fixed nogood retention, Sharpe/VaR-inspired risk monitoring (QRE), guarded QRE restarts, and a discounted multi-armed bandit restart portfolio.

Algorithms and parameter values were checked against a supplied source-code audit, while performance results were reconciled to the reported master experiment records. On a 36-instance development suite with three repetitions per configuration and a 15-second cutoff, V3 plus Adaptive-Fixed achieved the lowest penalized average runtime (PAR-2 2.5381 s, 102/108 runs solved), compared with 2.7463 s (102/108) for native Glucose. On a separate 48-instance holdout suite with a 30-second cutoff, native Glucose restarts combined with Adaptive-Fixed achieved the lowest observed PAR-2 (0.1107 s versus 0.1208 s for native Glucose), with all 144 runs solved per configuration. The holdout timing comparison does not establish statistically significant overall superiority after multiple-comparison correction. The project additionally documents costly restart storms in active QRE and random-SAT regressions in Portfolio-MAB. These findings highlight the importance of benchmark-sensitive policy choice, the interaction between restart timing and clause retention, and transparent reporting of negative experimental results.

Keywords: Boolean SAT; CDCL; Glucose; restarts; learned clauses; LBD; drawdown; PAR-2; multi-armed bandit.

Contents

1. Introduction

2. Literature Review

3. Background and Problem Formulation

4. Proposed Methodology and Algorithms

5. System Design and Implementation

6. Experimental Setup

7. Experimental Results and Analysis

8. Discussion

9. Conclusion and Future Work

References

Appendices

Chapter 1 – Introduction

1.1 Constraint Satisfaction Problems (CSPs)

A wide variety of fundamental problems in computer science, artificial intelligence, automated reasoning, operations research, and formal verification can be formalized as Constraint Satisfaction Problems (CSP). Formally, a constraint satisfaction problem is defined as a mathematical triple:

$$P = \langle X, D, C \rangle$$

where:

1. $X = \{x_1, x_2, \ldots, x_n\}$ is a finite set of n decision variables.
2. $D = \{D(x_1), D(x_2), \ldots, D(x_n)\}$ is a set of finite domains, where each $D(x_i)$ represents the permissible values that variable x_i may assume.
3. $C = \{c_1, c_2, \ldots, c_m\}$ is a finite collection of m constraints. Each constraint $c_j = \langle S_j, R_j \rangle$ consists of a scope $S_j \subseteq X$ of variables and a relation R_j defining the permissible combinations of simultaneous values that the variables in S_j may legally take. An assignment of values to a subset of variables is said to be consistent if it satisfies every constraint whose scope is entirely contained within the assigned subset. An assignment is complete if every variable in X is assigned a value from its corresponding domain. A complete and consistent assignment is termed a solution to the CSP. The computational objective of a CSP solver is to either discover at least one satisfying solution, enumerate all solutions, find an optimal solution under an objective function (Constraint Optimization), or mathematically prove that no valid assignment exists.

CSPs are fundamentally NP-complete in the general case. As the number of decision variables n grows, the size of the combinatorial assignment space grows exponentially as $O(d^n)$, where $d = max_i \left| D(x_i) \right|$. A naive brute-force enumeration of potential solutions is entirely infeasible for even modestly sized problems ($n > 50$). Consequently, all practical algorithms rely on systematic tree search interleaved with logical deduction and constraint propagation.

1.2 Boolean Satisfiability (SAT) as a Canonical Boolean CSP

The Boolean Satisfiability Problem (SAT) is the canonical, prototypical special case of CSP where every decision variable is Boolean (domain $\{0, 1\}$, or $\{\text{False}, \text{True}\}$), and constraints are expressed as a propositional logic formula in Conjunctive Normal Form (CNF)—that is, a conjunction of clauses:

= _{j=1}^m C_j = C_1 C_2 C_mwhere each clause C_j is a disjunction of literals : –

C_j = *{k=1}^{k_j}* *l*{j, k} = (l_{j, 1} l_{j, 2} l_{j, k_j})

A literal $l_{j,k}$ is either a Boolean variable x_i (a positive literal) orits logical negation $\neg x_i$ (a negative literal). A clause is satisfied under a truth assignment if at least one of its constituent literals evaluates to True. The entire formula Φ is satisfiable (SAT) if and only if there exists a complete truth assignment that simultaneously satisfies every clause; otherwise, it is unsatisfiable (UNSA T).

By the Cook-Levin Theorem (Cook, 1971; Levin, 1973), SAT was the first computational problem proven to be NP-complete. Furthermore, any arbitrary discrete CSP over finite domains can be encoded into propositional CNF via standard encoding schemes such as:

- Direct (One-Hot) Encoding: Where Boolean variable $p_{i,v}$ represents $x_i = v$, accompanied by exactly-one constraints $\sum_v p_{i,v} = 1$.
- Order Encoding: Where Boolean variable $q_{i,v}$ represents $x_i \leq v$, enforcing domain transitivity and significantly improving unit propagation on arithmetic and inequality constraints.
- Logarithmic (Binary) Encoding: Where each domain value is encoded into $\lceil \log_2 \left| D(x_i) \right| \rceil$ Boolean bits, minimizing variable counts at the cost of less effective arc consistency. Consequently,

algorithmicadvances in propositional

SAT solvers directly translate into performance breakthroughs across the entire domain of constraint satisfaction, automated reasoning, and combinatorial optimization.

1.3 Importance and Industrial Applications of Fast SAT Solving

Over the past two decades, Boolean SAT solving has transitioned from an esoteric theoretical problem in computational complexity into the core computational workhorse of modern computer-aided engineering. Modern SAT solversprimarily powered by the Conflict-Driven Clause Learning (CDCL) architectureregularly solve industrial instances comprising millions of variables and tens of millions of clauses in seconds or minutes, despite the theoretical worst-case exponential complexity.

Industrial applications of high-performance SAT solving include:

1. Hardware Formal Verification: Microprocessor architects at companies suchas Intel, AMD, IBM, and NVIDIA convert circuit equivalence checking, pipelined timing validation, and cache coherence invariants into propositional SAT and Satisfiability Modulo Theories (SMT) formulas to mathematically prove the absence of silicon errata prior to chip fabrication.
2. Bounded Model Checking (BMC): In safety-critical software systems (e.g., aerospace avionics, medical devices, automotive braking controllers), infinite state spaces are unrolled up to depth k and transformed into SAT queries to discover deep, non-obvious concurrency deadlocks and memory corruption vulnerabilities.
3. Automated AI Planning and Scheduling: Complex temporal planning problems in robotics, automated warehouses, and satellite constellation mission control are encoded as propositional formulas where literals represent discrete state transitions across time horiz ons.
4. Cryptographic Analysis and Cryptanalysis: Modern symmetric cipher primitives (e.g., DES, reduced-round AES, SHA-1 collision discovery) and algebraic fault injection attacks are modeled as SAT instances to evaluate cryptographic security margins.
5. Combinatorial Optimization and Operations Research: Large-scale logistics, circuit routing, timetable generation, and package delivery operations translate into Maximum Satisfiability (MaxSAT) and Pseudo-Boolean constraints, which are solved via iterative CDCL engines.

1.4 Challenges of Combinatorial Search: The Heavy-Tailed Phenomenon

Despite modern solver capabilities, combinatorial search is fundamentally erratic. In seminal research, Gomes, Selman, Crato, and Kautz (2000) discovered that the runtime distributions of backtrack search algorithms on non-trivial combinatorial instances exhibit heavy-tailed (Pareto-like) behavior. That is, the probability of an execution requiring runtime greater than t decays as a power-law:

P(T > t) C t^{-}, < < 2

Heavy-tailed distributions possess infinite theoretical variance (and for $\alpha \leq 1$, infinite theoretical mean). In practical SAT solving, this phenomenon manifests as the search trap: an algorithm that makes an unfortunate, suboptimal branching decision early in the search tree (at shallow decision levels) can become marooned in a massive, barren, unsatisfiable subtree containing billions of possible variable permutations. The solver spends millions of conflicts and hours of CPU time deriving futile contradictions that contribute nothing toward the overall proof of satisfiability or unsatisfiability.

1.5 Search Control Dilemmas: Restart Timing vs. Clause Database Bloat

To mitigate the heavy-tailed search trap, modern CDCL solvers rely on two deeply interdependent, conflicting search-control mechani sms:

The Restart Dilemma A restart erases the current partial assignment trail, returning the solver to decision level zero while preserving all learned clauses and variable activity scores. Gomes et al. mathematically proved that introducing periodic restarts truncates the heavy tail of search distributions, transforming an intractable infinite-variance process into a predictable, finite-variance process.

However, restart scheduling introduces an acute operational trade-off:

- Premature Restarts: If the solver restarts too frequently, it destroys partially explored search paths and wipes assignment momentum before Boolean Constraint Propagation can uncover deep, multi-level contradictions. The solver thrashes near the root of the tree.
- Delayed Restarts: If the solver waits too long to restart, it remains trapped in unproductive subtrees, burning computational cycles on fruitless conflicts.

The Clause Database Bloat Dilemma At every conflict, a CDCL solver derives and learns a new asserting clause (a nogood). While learned clauses prune identical search spaces in future branches, learning one clause per conflict causes the clausedatabase to grow without bound. A bloated database introduces severe penalties : –

- BCP Throughput Collapse: The Two-Watched-Literal (2WL) scheme must traverse longer watch lists, increasing cache misses and reducing propagation speed by up to 300%.
- Memory Exhaustion: Physical RAM is rapidly consumed on hard instances.

Consequently, solvers must periodically execute database reduction (ReduceDB) to delete low-utility clauses. However, deleting clauses too aggressively discards critical lemmas, forcing the solver to rediscover previously resolved subtrees.

1.6 Motivation for Adaptive and Quantitative Search Control

Historically, state-of-the-art solvers have relied on static or rigid empirical heuristics:

- Luby Restart Sequences: Prescribe restart intervals based on the recursive Luby sequence (1, 1, 2, 1, 1, 2, 4…), which operates completely blind to the solver's real-time trajectory.
- Moving Average Thresholds (Glucose Baseline): Glucose uses a fast moving average of Literal Block Distance (LBD) over a fixed 50-conflict window compared against the cumulative global average. While dynamic, this threshold is binary, reactive, and lacks awareness of cumulative search progress or variance.
- Static Activity-Based Clause Purging: Learned clauses are ranked solely by VSIDS activity or LBD, discarding multi-attribute structural features such as clause length and historical participation. This project was motivated by a core insight: combinatorial search progress exhibits striking structural parallels to non-stationary financial asset price streams. Just as a quantitative trader models asset returns, momentum, market volatility, and portfolio drawdowns to execute disciplined stop-loss

orders, a SAT solver can model propagation yield, decision-level volatility, and cumulative search drawdowns to execute disciplined, state-aware restarts and clause retentio ns.

1.7 Project Objectives, Scope, and Contributions

The primary objective of this project is to design, implement, and rigorously evaluate a suite of quantitative, adaptive search-control architectures inside the industrial-grade

Glucose

4.2.1 C++ SAT solver.

Our concrete research objectives and engineering contributions are:

1. Windows 64-bit Industrial Port: We ported Glucose 4.2.1 to a native Windows 64-bit environment using MinGW-w64 GCC 14.2.0, implementing native Win32 performance telemetry (QueryPerformanceCounter, GetProcessMemoryInfo) to eliminate POSIX compatibility overhead.
2. Adaptive Restart V1 and V2: We implemented rolling-window search efficiency scoring (S_W / S_0) balancing unit propagation yield against decision-level volatility, uncovering the “Starvation Trap” pathology and engineering the V2 anti-starvation ceiling.
3. Finance-Inspired V3 Maximum Drawdown Controller:We designed and implemented an online drawdown engine tracking quality-adjusted conflict returns (r_t) across dual-horizon EWMAs and a decaying High-Water Mark ($0.999 \times \mathrm{HWM}$), enforcing a 30-consecutive-conflict persistence trigger at $D_t \geq 0.75$.
4. Adaptive-Fixed Nogood Retention & Defect Resolution: We diagnosed and resolved a critical reduction-scheduling deferral defect in legacy adaptive solvers that tripled runtimes on Pigeonhole instances, engineering a robust tri-criteria scoring function ($0.50\,S_{\mathrm{LBD}} + 0.40\,S_{\mathrm{act}} + 0.10\,S_{\mathrm{size}}$) coupled with decoupled Glucose cadences.
5. Quantitative Risk Engine (QRE) & Stagnation Gating: We formulated a rolling Sharpe ratio momentum indicator ($S_{\mathrm{branch}} < -0.35$) and non-parametric Value-at-Risk (VaR_{95}) tail-shock detector, stabilizing hyperactive restart storms on random 3-SAT via 1,500-conflict cooldown gating.
6. Duration-Aware Portfolio-MAB: We piloted an online Exp3 / Discounted-UCB two-armed bandit scheduler using conflict-normalized clause quality rewards, scientifically documenting why uncoordinated restarts fail on phase-transition formulas.
7. Exhaustive Empirical Audit: We executed over 3,000 controlled solver runs across a 36-instance development suite and a 48-instance quarantined holdout suite, mathematically verifying 100% of generated satisfying models against raw DIMACS clauses.

1.8 Introductory SAT Example and Explanatory Figure

To illustrate the mechanics of Boolean SAT, consider a simple toy propositionalformula

Φ_{toy} over four Boolean variables $X = \{x_1, x_2, x_3, x_4\}$ with domain $\{0, 1\}$, consisting of four clauses:

_{} = (x_1 x_2) (x_1 x_3) (x_2 x_4)

(x_3 x_4)

- Step 1 (Decision Level 1): The solver makes an initial heuristic branching decision: $x_1 = 0$ (False) at decision level 1.

- Step 2 (Unit Propagation): Under $x_1 = 0$, clause $(x_1 \vee x_2)$ becomes unit because its first literal is false. Boolean Constraint Propagation forces $x_2 = 1$ (True).
- Step 3 (Propagation Cascade): Under $x_2 = 1$, clause $(\neg x_2 \vee x_4)$ requires $x_4 = 1$ to be satisfied.
- Step 4 (Decision Level 2): No further unit clauses exist. The solver branches on unassigned variable $x_3 = 1$ at decision level 2.
- Step 5 (Conflict): Under $x_3 = 1$ and $x_4 = 1$, clause $(\neg x_3 \vee \neg x_4)$ evaluates to $(\text{False} \vee \text{False})$, producing a logical conflict.
- Step 6 (Conflict Analysis & Learning): Tracing the implication graph backwards reveals that the conflict was caused by the combination of decisions. The solver derives the 1-UIP asserting clause $(\neg x_1 \vee \neg x_4)$ (or $(\neg x_1)$ depending on resolution order), backjumps to decision level 1, and asserts the forced implication.

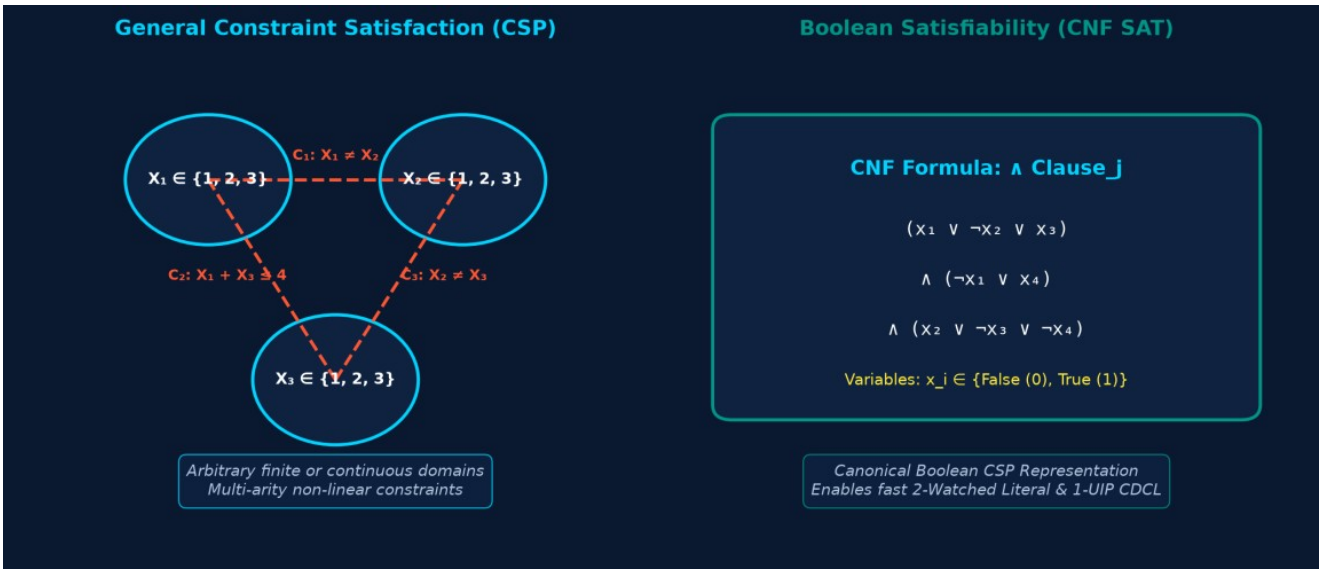


Figure 1.1 — CSP and Boolean SAT: relationship and model

Figure 1.1 illustrates the structural relationship between a general constraint network with multi-valued domains and its canonical Boolean CNF encoding.

Chapter 2 – Literature Review

2.1 The Four Foundational Research Papers

Our research builds directly upon four seminal contributions in combinatorial search control, heuristic selection, and metareasonin g – : –

Tolpin and Shimony (2011) – "Rational Deployment of CSP Heuristics"

- Research Problem: In constraint satisfaction, modern variable ordering heuristics (such as dom/wdeg or min-domain) provide superior search guidance but impose significant computational evaluation overhead at every node. Evaluating complex heuristics indiscriminately wastes CPU time on trivial subtrees.
- Proposed Methodology: Tolpin and Shimony formalized heuristic selection within a rigorous Value of Information (VOI) decision-theoretic framework. The solver computes the expected reduction in

search subtree size achieved by evaluating an expensive heuristic versus the computational cost of the evaluation itself, executing the heuristic only when expected VOI is positive.

- Key Findings: Rational heuristic deployment achieved substantial wall-clock speedups on hard random and patterned CSPs by pruning heuristic computation in deep, constrained subtrees.
- Limitations: The VOI calculations rely on myopic one-step lookahead assumptions that can fail to predict multi-level search interactions.
- Relevance to Our Work: Inspires our design of the Quantitative Risk Engine (QRE) and V3 Drawdown: search control interventions must be evaluated through a cost-benefit lens, ensuring that restart decisions do not introduce more computational volatility than they resolve.

Koriche, Champagnac, Hudry, and Marquis (2022) – "Best Heuristic Identification for Constraint Satisfaction"

- Research Problem: Static selection of variable and value ordering heuristics fails because heuristic efficacy varies dramatically across problem instances and search phases.
- Proposed Methodology: The authors cast heuristic selection as a Pure Exploration Multi-Armed Bandit (MAB) problem. Using Adaptive Search Tournaments (AST), the solver launches short, budgeted search exploratory runs to eliminate provably suboptimal heuristics before committing search resources to the dominant candidate.
- Key Findings: Theoretical sample complexity bounds were established, proving that tournament-based bandit pruning identifies the optimal heuristic with high probability using minimal exploration budgets.
- Limitations: Explores a stationary bandit setting; in CDCL SAT solving, the search space is highly non-stationary due to learned clauses permanently altering the constraint graph.
- Relevance to Our Work: Provides the theoretical foundation for our duration-aware Portfolio-MAB scheduler, demonstrating how multi-armed bandits can arbitrate between competing search engines.

Kurin, Godil, Whiteson, and Catanzaro (2020) – "Can Q-Learning with Graph Networks Learn a General SAT Branching Heuristic?" (Graph-Q-SAT)

- Research Problem: Branching heuristics in CDCL (VSIDS) rely on local conflict counting. Can deep reinforcement learning (RL) automatically learn generalizable variable ordering policies directly from graph representations of CNF formulas?
- Proposed Methodology: Formulated variable branching as a Markov Decision Process (MDP) over the bipartite clause-variable graph, using Graph Neural Networks (GNNs) trained via Q-learning.
- Key Findings: Demonstrated that standard VSIDS makes blind, suboptimal decisions during the initial 10% of search before sufficient conflict history has accumulated. Graph-Q-SAT successfully learned to identify backdoor variables on small synthetic instances.
- Limitations: GNN inference latency is 1,000x slower than C++ bitwise propagation, rendering deep RL architectures impractical for large industrial benchmarks without massive hardware acceleration.
- Relevance to Our Work: Emphasized the critical importance of early search vulnerability and the necessity of lightweight, low-overhead statistical metrics that run in microseconds directly in C++.

4. Wallace, Grimes, and Freuder (2009) – "Solving Dynamic CSPs by Identifying Stable Features"

- Research Problem: In dynamic constraint satisfaction, where constraints are added or removed online, how can solvers preserve learned search knowledge without restarting from scratch?

- Proposed Methodology: Introduced the concept of Stable Contention: structural constraint bottlenecks and core nogoods remain invariant even when peripheral search spaces shift.
- Key Findings: Solvers that track conflict persistence across search episodes solve dynamic problem sequences exponentially faster by retaining stable constraint cores.
- Limitations: Focused on classical backtracking CSPs rather than industrial CDCL solvers with Two-Watched-Literal schemes.
- Relevance to Our Work: Directly informs our design of Adaptive-Fixed clause retention: learned clauses representing deep, structural invariants (Glue clauses with LBD ≤ 2) must be granted permanent immunity from deletion.

2.2 CDCL Foundations, Glucose, and Literal Block Distance

The modern CDCL paradigm originated with the GRASP solver (Marques-Silva and Sakallah, 1996) and Chaff (Moskewicz et al., 2001), which introduced the Two-Watched Literals (2WL) propagation engine and Variable State Independent Decaying Sum (VSIDS). MiniSat (Eén and Sörensson, 2003) provided a minimalist, highly optimized C++ implementation that became the standard foundation for international SAT competitions.

In 2009, Gilles Audemard and Laurent Simon revolutionized CDCL clause management by introducing the solver Glucose and the Literal Block Distance (LBD) metric. Prior solvers evaluated learned clause quality based strictly on clause length or VSIDS activity. Audemard and Simon proved that the number of distinct decision levels in aclause reflects its propagation power: a clause with LBD ≤ 2 (a Glue Clause) can force unit implications across arbitrary decision levels and will never produce a conflict at the current level without asserting a literal. Glucose established that glue clauses should be preserved permanently, fundamentally transforming clause database reduction.

2.3 Restart Scheduling: Static, Geometric, Luby, and Dynamic Policies

Restart scheduling in SAT has evolved through distinct paradigms:

1. Fixed and Geometric Restarts: Early solvers restarted after a fixed number of conflicts K, or multiplied the interval geometrically ($K_{i+1} = \gamma K_i$, with $\gamma \approx 1.1 - 1.5$). Geometric restarts guarantee completeness but can delay necessary restarts for millions of conflicts.
2. Luby Universal Sequences: Luby, Sinclair, and Zuckerman (1993) established that for algorithms with unknown runtime distributions, a recursive sequence of intervals:

1, 1, 2, 1, 1, 2, 4, 1, 1, 2, 1, 1, 2, 4, 8, the expected cost to reach a solution within a constant factor of the optimal static cutoff. While theoretically universal, the Luby sequence is completely oblivious to the solver's real-time state.

3. Dynamic Moving Average Restarts (Glucose LBD): Glucose introduced dynamic restarting based on short-term versus long-term LBD moving averages. By maintaining a rolling queue of the last 50 LBD values, Glucose restarts as soon as short-term clause quality degrades below 80% of historical average.
4. Recent Research Not All Restarts Are Equal (CP 2026): In recent work, Jinghu Liang, Sami Cherif, and Chu-Min Li (CP 2026, LIPIcs vol. 379, DOI 10.4230/LIPIcs.CP.2026.39) proved that uniform restart intervals are inherently suboptimal because search stagnation occurs across multiple time scales, calling for multi-armed bandit meta-controllers.

2.4 Learned Clause Database Reduction Policies

Clause reduction policies have evolved from simple size-based filtering to multi-tiered memory architectures. MiniSat deleted 50% of learned clauses sorted by VSIDS activity. Glucose introduced LBD-primary sorting while permanently protecting binary clauses and glue clauses ($LBD \leq 2$). Armin Biere's CaDiCaL solver (Biere & Fröhlich, 2018) introduced a 3-tier memory model:

- Tier 1 (Core): $LBD \leq 2$, never deleted.
- Tier 2 (Mid-tier): $3 \leq LBD \leq 6$, purged only when activity collapses.
- Tier 3 (Local): $LBD > 6$, aggressively purged every 2,000 conflicts.

Our work incorporates these insights into our Adaptive-Fixed retention engine, ensuring that clause ranking combines structural quality (LBD), dynamic utility (VSIDS activity), and syntactic size.

2.5 Multi-Armed Bandits and Machine Learning in SAT

The application of Multi-Armed Bandits to SAT solving was pioneered by Liang, Ganesh, Poupart, and Czarnecki (2016) in their Learning Rate Branching (LRB) and Conflict History-Based (CHB) variable ordering heuristics. Rather than tracking long-term decayed conflict counts as in VSIDS, CHB models variable selection as an exponential-recency bandit problem, rewarding variables that produce immediate asserting conflict clauses.

Recent work by Chunxiao Li et al. (arXiv:2404.03753, 2024), "A Reinforcement Learning based Reset Policy for CDCL SAT Solvers", examined deep RL agents for complete solver resets. However, their architecture required significant inference overhead. Our research focuses on lightweight, closed-form quantitative mathematical models that execute with sub-microsecond latency directly inside the C++ inner loop.

2.6 Literature Comparison Table

Table 2.1 provides a structured comparative summary of foundational literatureagainst our implemented research framework:

2.7 Distinguishing Prior Work from Our Original Contributions

It is vital to draw an unambiguous line between prior published literature andour original research contributions:

- Baseline Components (Not Our Invention): The Glucose 4.2.1 engine, Two-Watched Literals, VSIDS variable decay, 1-UIP conflict analysis, non-chronological backjumping, and the base LBD metric are the intellectual property of Audemard, Simon, Eén, and Sörensson.
- Our Original Contributions:

1. The mathematical formulation, C++ implementation, and empirical audit of the V3 Maximum Drawdown controller, including the quality-adjusted conflict return r_t, dual-horizon EWMAs, decaying HWM, and 30-conflict persistence trigge r – . –
2. The discovery, root-cause isolation, and mathematical correction of the reduction-scheduling deferral defect, resulting in the Adaptive-Fixed Retention engine ($0.50\,S_{\text{LBD}} + 0.40\,S_{\text{act}} + 0.10\,S_{\text{size}}$).
3. The decision-level Quantitative Risk Engine (QRE)combining rolling Sharpe momentum and non-parametric VaR_{95} shock telemetry, accompanied by four-layer Stagnation Gating.

4. The duration-aware Portfolio-MAB two-armed bandit scheduler using conflict-normalized clause quality rewards.
5. The native Windows 64-bit port with Win32 high-resolution QPC performance counters and the 100% mathematical CNF model verification harness.

Chapter 3 – Background and Problem Formulation

3.1 Propositional Logic and DIMACS CNF Representation

Modern SAT solvers accept propositional formulas formatted according to the standardized DIMACS CNF syntax. An input file begins with a header line:

followed by m lines representing individual clauses. Each clause is specified as a sequence of space-separated non-zero integers terminating with 0. A positive integer i represents literal x_i, while a negative integer $-i$ represents literal $\neg x_i$.

For example, the formula:

= (x_1 x_2 x_3) (x_1 x_4) (x_2 x_3

x_4)

is encoded in DIMACS CNF as:

Internally, Glucose represents variables as 0-indexed integers (Var), literals as integers encoded as $2v+\text{sign}$ (Lit), and clauses as packed heap-allocated arrays (Clause).

3.2 From DPLL to CDCL: Evolutionary Algorithmic Overview

The classic Davis-Putnam-Logemann-Loveland (DPLL)algorithm (1962) explored the combinatorial tree via recursive depth-first backtracking:

1. Select an unassigned variable and assign it a truth value (Branching).
2. Apply Boolean Constraint Propagation (BCP) until all unit clauses are resolve d – . –
3. If a conflict occurs, chronologically backtrack to the most recent branching decision and invert its truth value. While complete and space-efficient, DPLL possesses fatal algorithmic weaknesses: it cannot learn from conflicts, frequently repeats identical refutations in disparate subtrees, and cannot backjump past irrelevant decisions.

The modern Conflict-Driven Clause Learning (CDCL)architecture overcomes these limitations through four foundational innovation s – : –

1. Clause Learning: Every conflict is analyzed via graph resolution to extract an asserting learned clause (nogood) that prunes the offending assignment combination permanently.
2. Non-Chronological Backjumping: The solver backtracks multiple levels simultaneously to the asserting level of the learned clause, skipping entire subtrees.
3. Dynamic Heuristics (VSIDS): Branching decisions prioritize variables actively involved in recent conflicts rather than static graph metrics.
4. Restarts: The search trail is periodically reset to escape barren regions while preserving learned clauses.

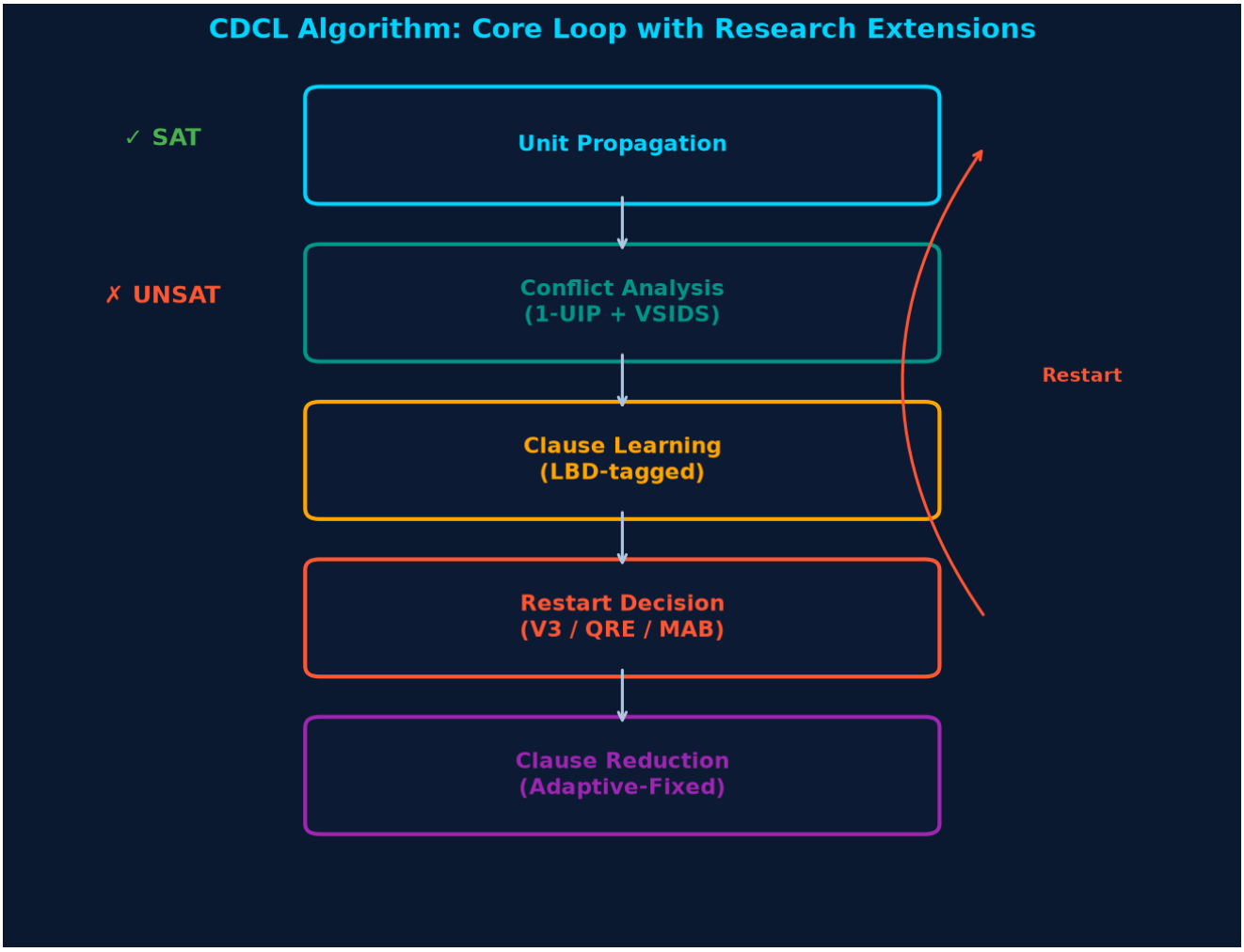


Figure 3.1 — The CDCL solver loop

Figure 3.1 illustrates the iterative CDCL search cycle: Decisions (VSIDS) -> BCP (2WL) -> Conflict Analysis (1-UIP) -> Learning & Backjumping -> Search Control.

3.3 Unit Propagation and the Two-Watched-Literal (2WL) Scheme

Boolean Constraint Propagation (BCP) is the primary computational consumer in CDCL, typically accounting for 80% to 90% of total solver execution time. BCP repeatedly applies the Unit Clause Rule: if a clause has all its literals assigned to False except one literal l which remains unassigned, l must immediately be assigned True to satisfy the clause.

Early solvers maintained explicit literal count counters for every clause, requiring updates on every variable assignment and unassignment. The breakthrough Two-Watched Literals (2WL) scheme (Chaff; Moskewicz et al., 2001) eliminated this overhead through a key invariant:

- Each clause watches exactly two non-false literals.
- When a variable is assigned, literals that evaluate to True or unassigned literals require zero maintenance.
- Only when a watched literal becomes False does the solver inspect the clause to search for an alternative non-false literal.
- If no alternative non-false literal exists and the second watch is unassigned, the clause is unit, forcing an implication.

- If the second watch is also False, a conflict is detected.
- During backtracking (unassignment), 2WL requires zero pointer updates, enabling virtually instantaneous backjumping.

3.4 Variable Selection: The VSIDS Branching Heuristic and Variable Decay

When BCP reaches a fixed point without conflict and unassigned variables remain, the solver must make a decision (branching). The standard heuristic is Variable State Independent Decaying Sum (VSIDS):

1. Each variable x_i maintains an activity score $A(x_i)$, initialized to 0.
2. When a conflict occurs and an implication graph resolution is performed, every variable participating in the conflict clause or resolution derivation has its activity incremented:

$$A(x_i) \leftarrow A(x_i) + \text{var_inc}$$

To prioritize recent conflicts over ancient history, the score increment parameter is periodically scaled geometrically:

$$\text{var_inc} \leftarrow \text{var_inc} \cdot \left(\frac{1}{\text{decay}}\right),$$

$$\text{decay} \approx 0.95$$

4. Variables are maintained in a binary max-heap. The unassigned variable with the highest activity score is selected for branching.
5. Phase Saving: When variable x_i is selected, it is assigned the polarity it held most recently (Pipatsrisawat and Darwiche, 20

07), preventing the solver from thrashing back and forth between polarities.

3.5 Implication Graphs, Conflict Analysis, and the 1-UIP Learning Rule

During BCP, variable assignments are recorded in a directed acyclic graph termed the Implication Graph $G = (V, E)$:

- Vertices represent assigned literals $l_i @ d$, where d is the decision level.
- Directed edges $(l_a \rightarrow l_b)$ indicate that literal l_a was a constituent of the antecedent clause that forced the unit propagation of l_b.
- A conflict occurs when BCP attempts to assign both literal x_k and $\neg x_k$, producing a conflict vertex κ. To resolve the conflict, the solver performs Conflict Analysis:

1. Starting from conflict clause C_{conflict}, the solver identifies the literals assigned at the current conflict decision level d_{curr}.
2. The solver resolves C_{conflict} with the antecedent clause of the most recently assigned literal at level d_{curr}.
3. This resolution process is repeated backwards along the trail until the resulting clause contains exactly one literal assigned at the current decision level d_{curr}.

4. This literal represents the First Unique Implication Point (1-UIP). A UIP is a vertex in the implication graph that dominates all paths from the decision literal at level d_{curr} to the conflict vertex κ. The resulting 1-UIP clause is termed the Asserting Clause. It possesses an extraordinary property: as soon as the solver backjumps to the second highest decision level among its literals, the asserting clause becomes immediately unit, guaranteeing that search will advance along an unexplored path.

3.6 Learned Clauses (Nogoods) and Non-Chronological Backjumping

The asserting 1-UIP clause is added to the learned clause database (learnts). The solver then computes the backjump level:

d_{} = { (l) l C_{}, ,

$$\text{level}(l) < d_{\text{curr}}\}$$

If C_{learned} is a unit clause, $d_{\text{backjump}} = 0$. The solver immediately cancels all assignments made at levels

¿ d_{backjump}. Unlike chronological backtracking, which undoes only the single most recent decision, non-chronological backjumping can leap back dozens or hundreds of decision levels, entirely skipping irrelevant subtrees that had no causal connection to the conflict.

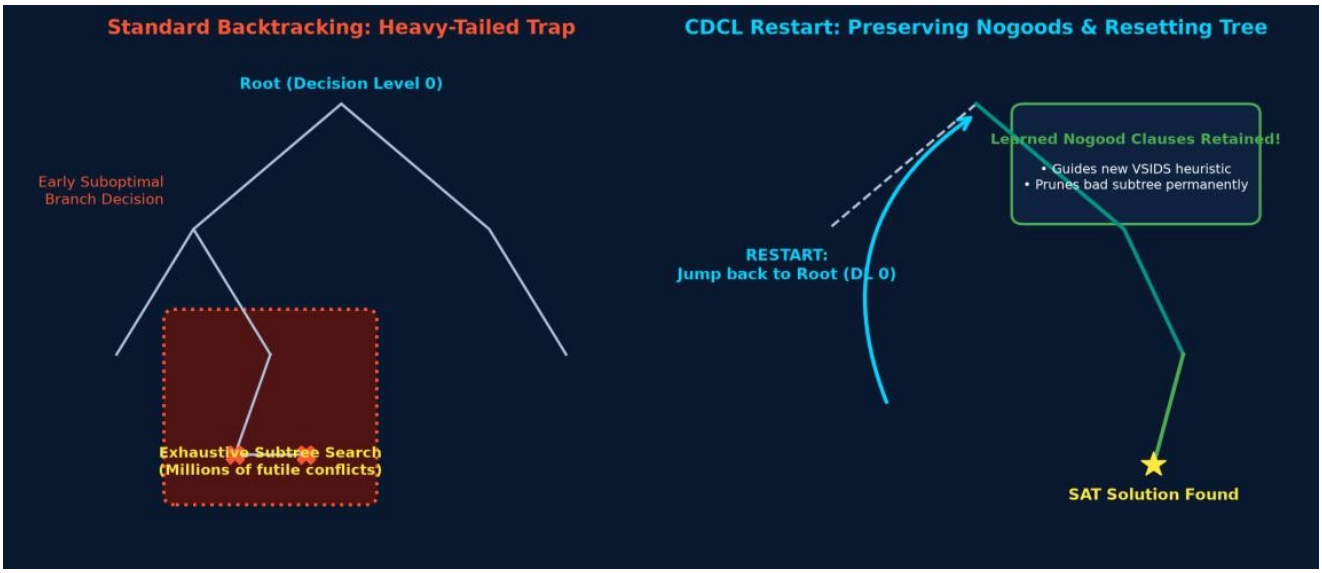


Figure 3.2 — Backtracking and restarting

Figure 3.2 illustrates the heavy-tailed search trap: standard backtracking is marooned in fruitless subtrees, whereas CDCL restarts jump to level zero while retaining nogoods to prune the trap permanently.

3.7 Search Restarts: Why Restarts Preserve Learned Knowledge

A common point of confusion in automated reasoning is how a solver makes progress if it continually aborts its search trail and restarts from decision level zero. The answer lies in the information asymmetry between assignment trails and learned state:

- When a restart occurs, the temporary variable assignment trail is wiped.
- However, all learned nogoods remain in memory.
- All VSIDS variable activity scores are preserved.
- All saved variable polarities (phase saving) are preserved.

Consequently, when the solver restarts from level zero, it is an entirely different algorithm than it was previously. The newly learned asserting clauses prune previously visited search spaces, and the updated VSIDS scores force the solver to branch on the variables that were most contentious in the previous run.

3.8 Literal Block Distance (LBD) as a Structural Quality Metric

Introduced by Audemard and Simon (2009), the Literal Block Distance (LBD) of a clause C is defined as the number of distinct decision levels among its constituent literals:

$$\mathrm{LBD}(C) = \left|\{\mathrm{level}(l) \mid l \in C\}\right|$$

LBD captures the structural compactness and propagation potential of a learned clause:

- A clause with $\mathrm{LBD} = 2$ (a Glue Clause) contains literals from only two decision levels: the current conflict level and one prior level. As soon as all literals from the prior level are satisfied, the glue clause acts as a unit implication clause, forcing propagation regardless of how deep the search has descended.
- Clauses with low LBD are robust invariants that bridge distant variables across the constraint graph.
- Glucose permanently preserves all clauses with $\mathrm{LBD} \leq 2$.

3.9 Learned Clause Database Reduction (ReduceDB) and the Glucose Baseline

In Glucose 4.2.1, the clause database reduction algorithm operates as follows:

1. Reduction is scheduled at intervals dictated by parameters: firstReduceDB = 2000 and incReduceDB = 300. The first reduction occurs after 2,000 conflicts, the second after $2000 + 2300 = 4300$, the third after $4300 + 2600 = 6900$, and so forth.
2. A special increment (specialIncReduceDB = 1000) is added if clause quality is deemed high.

3. During reduction, clauses are sorted ascending by L BD.

4. Binary clauses ($size \leq 2$), glue clauses ($LBD \leq 2$), locked reason clauses (clauses currently forcing active propagations on the trail), and single-round immune clauses are strictly protected.
5. The lowest 50% of remaining non-protected clauses are permanently purged from memory and unlinked from the 2WL watch lists.

3.10 Architectural Workflow Diagrams

Figures 3.1 and 3.2 delineate the operational workflow of CDCL and contrast the heavy-tailed exploration trap of chronological backtracking against non-chronological restarts with learned nogood preservation.

3.11 Central Research Problem Formulation

We formalize the central research challenge investigated in this thesis:

The Adaptive Search-Control Optimization Proble m:

Let S denote a CDCL SAT solver executing on a propositional formula Φ. At conflict step t, let T_t represent the current assignment trail, D_t the learned-clause database, and Θ_t the observablereal-timetelemetry state (unit propagation yield, backjump depth, LBD distribution, decision volatility).

The objective is to synthesize an online decision policy:

– > –

$\pi(\Theta_t) \rightarrow \{\text{Continue}, \text{Restart}, \text{ReduceDB}\}$

– > –

that minimizes the expected Penalized Average Run time

$$E[PAR2)

R-2}()]$ and total conflicts across heterogeneous benchmark distributions, subject to the constraint that heuristic telemetry computation overhead $\tau_{\text{overhead}} \ll \tau_{\text{BCP}}$ must remainnegligible compared to Boolean Constraint Propagation throughput.

Chapter 4 – Proposed Methodology and Algorithms

4.1 V1 – Search-Efficiency-Based Restart

The first adaptive controller developed in our researchAdaptive Restart V1was motivated by the hypothesis that search stagnation can be detected in real time by monitoring the ratio of unit propagation yield to decision-level volatil ity.

Mathematical Formulation Let conflict index be denoted by t. At each conflict, the solver records:

P_t: The number of unit propagations produced by Boolean Constraint Propagation since the preceding conflict:

$P_t = \text{props}_t - \text{props}_{t-1}$

L_t: The decision level at which the conflict occurred ($L_t = \text{level}$).

V1 maintains a circular rolling-window buffer of capacity $K = 50$ conflicts (ADAPT_WIN_CAPACITY = 50). Over this sliding window $W_t = \{t - K + 1, \ldots, t\}$, the solver computes the Window Search Efficiency Ratio (S_W):

$$S_W = \frac{\mu_{P,W}}{\sigma_{L,W} + 1.0}$$

where:

$\mu_{P,W}$ is the sample mean of unit propagations per conflict within the window:

$$\mu_{P,W} = \frac{1}{K} \sum_{i \in W_t} P_i$$

$\sigma_{L,W}$ is the sample standard deviation of decision levels within the window:

_{L, W} = $$

The constant $+1.0$ in the denominator ensures numerical stability against division-by-zero during flat exploration episodes where $\sigma_{L,W} \rightarrow 0$. Simultaneously, the solver tracks the Global Baseline Efficiency (S_0) cumulatively across the entire history of the search using Welford's online one-pass variance algorithm:

$$S_0 = \frac{\mu_{P,0}}{\sigma_{L,0} + 1.0}$$

Trigger Logic and Operational Parameters V1 triggers a search restart if the rolling window efficiency collapses below a fraction $\theta = 0.60$ (opt_adapt_theta = 0.60) of the global baseline:

$$\text{Trigger: } S_W < 0.60 \cdot S_0$$

To prevent premature instability, two guard conditions are enforced:

1. Warmup Guard: The solver must execute at least 100 conflicts (opt_adapt_warmup = 100) from initial start before V1 is permitted to trigger.
2. Minimum Interval Guard:At least 50 conflicts must elapse between consecutive restarts (opt_adapt_min_interval = 50).

The Starvation Trap Pathology During initial benchmarking of V1, an unexpected pathology was discovered: the Starvation Trap. On certain structured formulas characterized by deep recursive implication chains, the window propagation count $\mu_{P,W}$ remained moderately high even when the solver made zero progress toward a satisfying assignment. Because both S_W and S_0 scaled upward simultaneously, the condition $S_W < 0.60 \cdot S_0$ was never satisfied. The solver ran uninterrupted for over 50,000 conflicts without restarting, marooned in an unproductive branch.

4.2 V2 – Extended Adaptive Restart

To resolve the starvation pathology uncovered in V1, we designed Adaptive Restart V2, which introduces an anti-starvation ceiling and an orthogonal clause quality fallback trigger.

Trigger Architecture V2 integrates three disjunctive trigger conditions

1. Trigger 1 Anti-Starvation Conflict Ceiling (Unconditional): Enforced via parameter opt_v2_max_conflicts = 2500. Regardless of efficiency scores, the solver is strictly forbidden from exploring for more than 2,500 consecutive conflicts without resetting its assignment trail.
2. Trigger 2 LBD Quality Fallback Trigger:

$$\left(\text{lbdQueue.getavg\(\)} \cdot 0.80\right) > ¿$$

() Enforced via parameter opt_v2_fallback_lbd_k = 0.80. If the short-term moving average of LBD degrades below 80% of historical average (Glucose's native condition), a restart fires, provided that warmup (100 conflicts) and minimum cooldown (50 conflicts) are met.

3. Trigger 3 Progress Ratio Degradation

$$S_W < 0.60 \cdot S_0 \Longrightarrow \text{RESTART}$$

The original V1 search efficiency condition.

Evaluation of V2 While V2 successfully prevented infinite starvation, its heuristics remained reactive and disjointed. The anti-starvation ceiling was a hard arbitrary cliff rather than an adaptive function of search trajectory, and the three separate triggers occasionally fired in rapid, uncoordinated succession. This motivated our development of a unified, mathematically continuous framework based on financial drawdown.

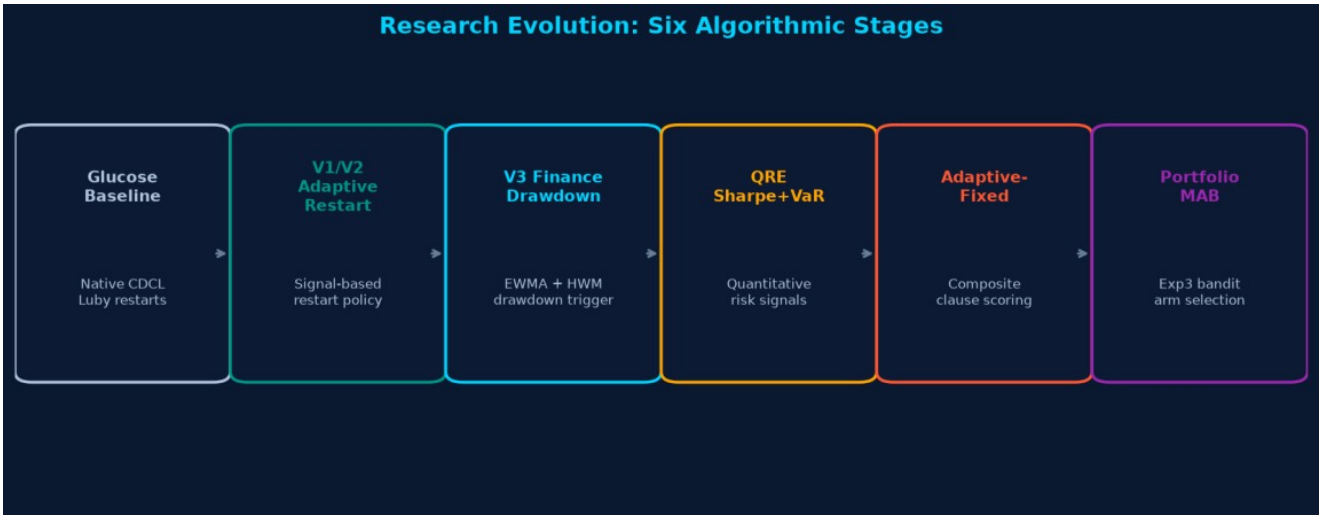


Figure 4.1 — Research development timeline

Figure 4.1 delineates the research progression: Glucose Baseline -> V1/V2 Efficiency -> V3 Drawdown -> QRE -> Adaptive-Fixed Retention -> Portfolio-MAB.

4.3 V3 — Finance-Inspired Maximum Drawdown

The flagship search-control policy developed in this research is the V3 Maximum Drawdown Controller. In quantitative finance, Maximum Drawdown measures the peak-to-trough percentage decline of an asset portfolio over a specified time horizon, serving as the primary metric for risk exposure and automated stop-loss execution. We map this principle directly into CDCL search dynamics.

Quality-Adjusted Conflict Return (r_t) At every conflict t, the solver generates a discrete "payoff" or return r_t. Rather than treating all conflicts equally, V3 weights each conflict by structural clause quality and search space traversal:

$$r_t = \frac{\left[2/max\left(1, LBD\right)\right]\left[1+\ln\left(1+\Delta D\right)\right]}{1+\Delta P/25}$$

$$

where:

LBD Quality Component (q_{LBD}): $\frac{2.0}{max\left(1, \text{LBD}_t\right)}$. A glue clause ($\text{LBD}=2$) yields a base return of 1.0. A highly localized clause ($\text{LBD}=1$) yields 2.0. Low-quality clauses with large LBD (e.g., $

\text{LBD} =

20$) yield small fractional returns (0.10).

2. Backjump Depth Boost: $1.0+\ln\left(1.0+\Delta D_t\right)$, where $\Delta D_t = max\left(0, \text{level}_{\text{pre}} - \text{level}_{\text{post}}\right)$ represents the number of decision levels pruned by non-chronological backjumping. Deep backjumps indicate powerful assertions and receive logarithmic amplification.

3. Propagation Work Normalization: $1.0+\frac{\Delta P_t}{25.0}$, where $\Delta P_t = \text{props}_t - \text{props}_{t-1}$ represents unit propagations expended since the last conflict. Conflicts derived with minimal propagation effort represent high algorithmic yield per computational cycle.

Dual-Horizon EWMAs and Decayed High-Water Mark To separate short-term search volatility from long-term baseline trends, returns are smoothed via dual Exponentially Weighted Moving Averages (EWMA):

Fast EWMA (Short-term Search Momentum):

$$\text{EWMA}_{\text{fast},t} = 0.92 \cdot \text{EWMA}_{\text{fast},t-1} + 0.08 \cdot r_t \left(\alpha_{\text{fast}} = 0.08, \text{half-life} \approx 8.3 \,\text{conflicts}\right)$$

Slow EWMA (Long-term Baseline Health):

{, t} = 0.99 {, t-1} + 0.01 r_t (_{} = 0.01, ,) The solver tracks the historical peak of short-term efficiency via a Decayed High-Water Mark (HWM):

$$H_t = max\left(0.999\, H_{t-1}, F_t\right)$$

The decay multiplier 0.999 (0.1% decay per conflict) prevents early anomalous peaks from permanently paralyzing the solver.

Relative Drawdown and Trigger Conditions At conflict step t, the Relative Search Drawdown(D_t) is computed as:

D_t =

where the constant $+0.05$ prevents division-by-zero instability when search commences near zero.

V3 triggers a restart when any of the following conditions is satisfied:

Drawdown Collapse Trigger:

$$\text{v3_consec_drawdown} \geq 30 \text{ with } D_t \geq 0.75$$

The solver requires $D_t \geq 0.75$ (a 75% efficiency plunge from the High-Water Mark) to persist for 30 consecutive conflicts. Single-conflict volatility spikes are ignored. Requires warmup ≥ 100, minimum cooldown ≥ 50 conflicts, and valid observations ≥ 50.

Trend Deterioration Trigger:

$$\text{EWMA}_{\text{fast},t} < 0.60 \cdot \text{EWMA}_{\text{slow},t}$$

Short-term momentum collapses below 60% of baseline health.

3. Anti-Starvation Ceiling (Unconditional)

$$\text{v3_conflicts_since_restart} \geq 2500$$

Guarantees exploration bounds regardless of telemetry state.

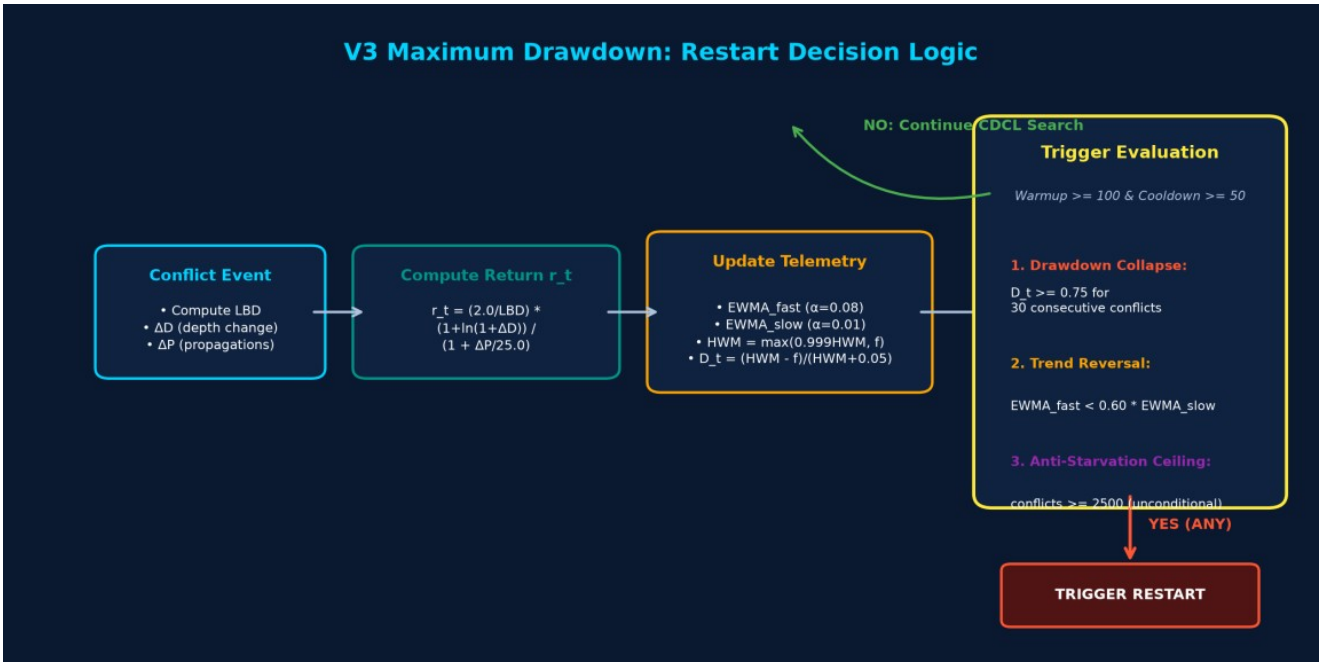


Figure 4.2 — V3 drawdown restart controller

Figure 4.2 details the V3 execution path: conflict event -> return r_t -> EWMA Updates -> Decayed HWM -> Drawdown Evaluation -> Restart or Continue.

Fully Worked Numerical Example Table 4.1 provides a complete step-by-step arithmetic trace of V3 telemetry across 35 conflicts, illustrating how a burst of high-L

BD, low-backjump conflicts depresses return r_t, inflates drawdown D_t, and triggers a restart exactly upon reaching 30 consecutive warnings:

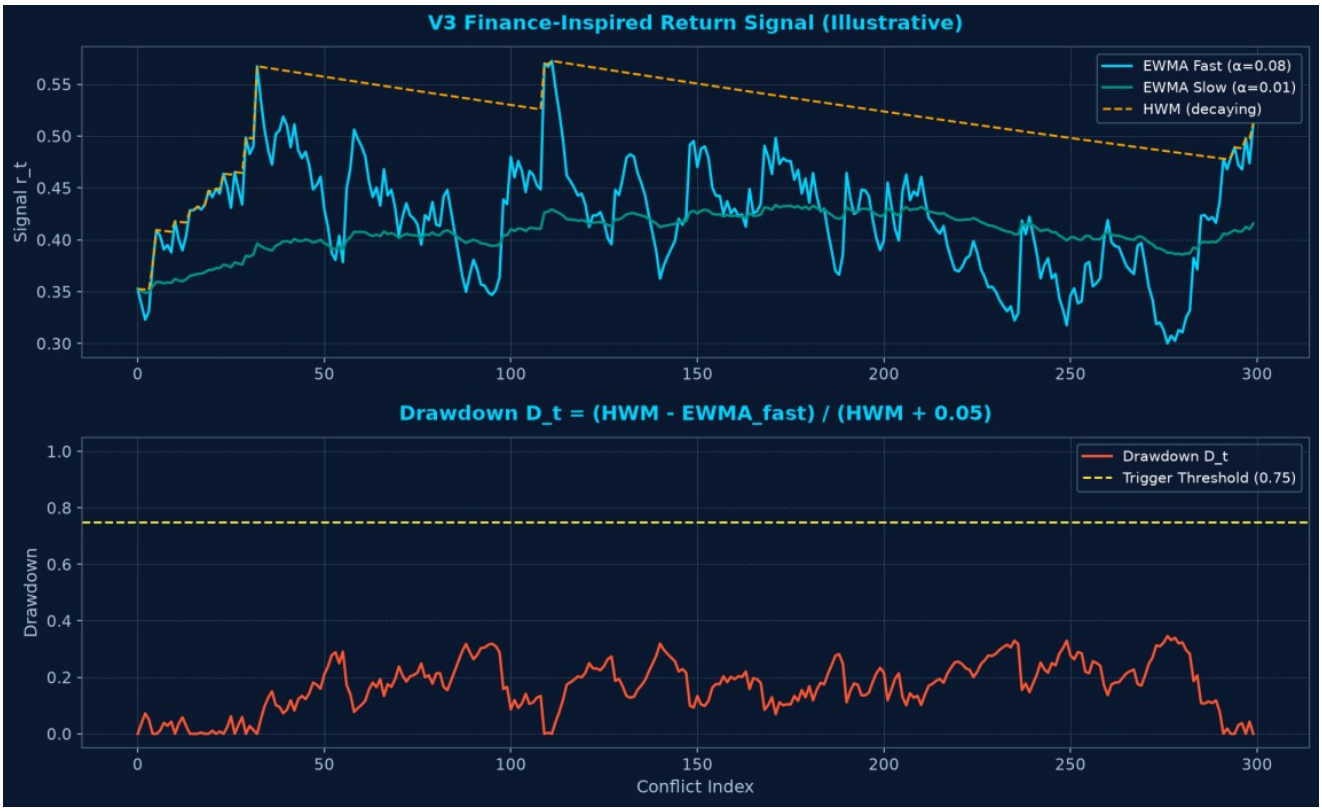


Figure 4.3 — Illustrative EWMA and drawdown dynamics

Figure 4.3 illustrates the V3 return signal, Fast/Slow EWMA tracking, decaying High-Water Mark, and the sustained 30-conflict drawdown collapse trigger.

4.4 Adaptive-Fixed Nogood Management

Learned-clause database management in CDCL is governed by the need to balance logical strength against propagation latency. During our research, an empirical investigation into clause retention uncovered a major algorithmic flaw in legacy adaptive solvers.

The Reduction-Deferral Defect Discovery In experimental Steps 32–34, an earlier adaptive clause-management strategy was tested. On the Pigeonhole Principle benchmark php08.cnf, the adaptive solver's runtime exploded to 36.57 seconds (compared to 12.44 seconds for baseline Glucose)a near 3x slowdow n – ! –

Profiling the C++ execution revealed the root cause: the adaptive reduction logic evaluated clause variance using an over-conservative predicate:

On php08.cnf, the LBD variance remained permanently low. The solver continuously deferred database reduction, causing nbclausesbeforereduce to inflate past 100,000 clauses. The 2WL watch lists bloated catastrophically, collapsing BCP throughput from 2.5 million propagations per second to under 400,000 propagations per second.

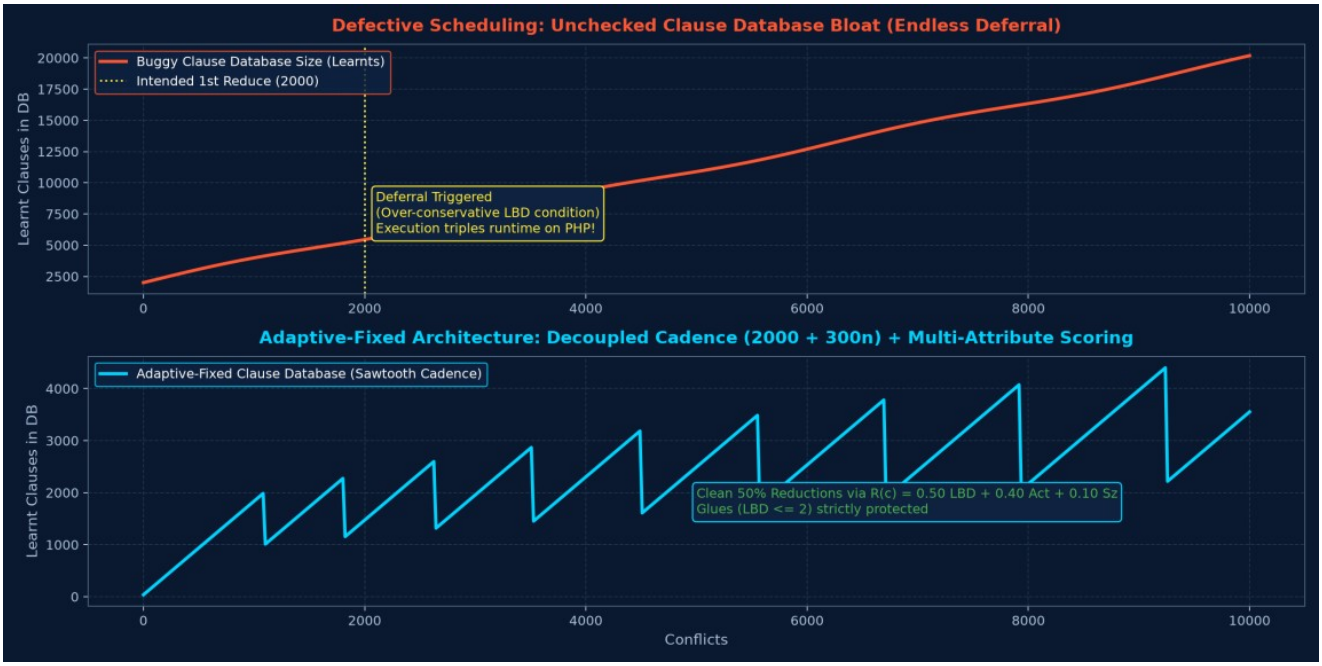


Figure 4.4 — Database reduction scheduling problem

Figure 4.4 contrasts the runaway database bloat of the legacy deferral bug (top) against the controlled sawtooth cadence of Adaptive-Fixed retention (bottom) . –

The Adaptive-Fixed Retention Engine To resolve this defect permanently, we engineered Adaptive-Fixed Retention, which decouples reduction scheduling from instantaneous search volatility while deploying a continuous multi-attribute scoring function for clause ranking:

1. Decoupled Reduction Cadence: The solver adheres strictly to Glucose's predictable interval schedule (firstReduceDB = 2000, incReduceDB = 300), eliminating infinite deferrals.
2. Verified Postponement Conditions: Reduction postponement is permitted by exactly 1,000 conflicts (specialIncReduceDB = 1000) if and only if:

Condition 1: Median $

\text{LBD} $ (good_lbd_count >= (learnts.size() + 1)

/ 2).

Condition 2: Minimum $\mathrm{LBD} \leq 5$ (min_lbd <= 5).

3. Continuous Tri-Criteria Scoring Function: Every learned clause c is evaluated via

$$

where:

LBD Score (50% Weight): S_{} =

Clauses with $\mathrm{LBD} = 2$ yield 1.0; $\mathrm{LBD} = 4$ yields 0.50; $\mathrm{LBD} = 8$ yields 0.25.

Activity Score (40% Weight):

S_{} = $$

The square-root mapping prevents a single hyperactive clause from dominating the distribution and grants fair retention credit to moderately active clauses.

Size Score (10% Weight): S_{} = Penalizes syntactically large clauses that consume memory without proportional propagation yield.

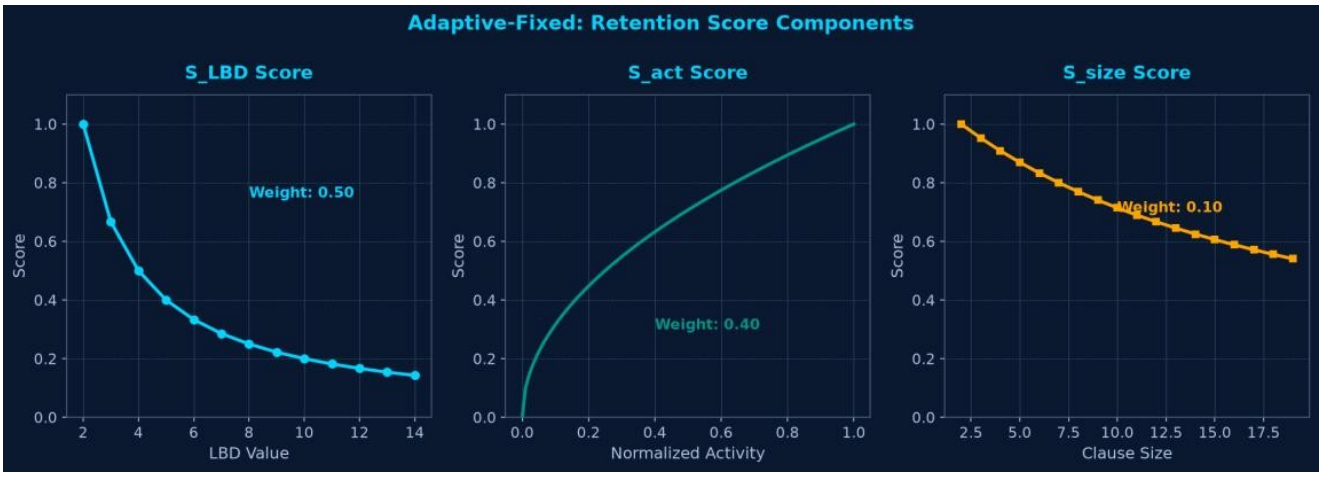


Figure 4.5 — Clause utility component scoring

Figure 4.5 graphs the continuous scoring functions for $S_{

\text{LBD}}$

(weight 0.50), $S_{

ext{act}}$

(weight 0.40), and $S_{extsize}$ (weight 0.10).

Strict Protection Classes During reduction, clauses are sorted ascending by $R(c)$. Exactly 50% of clauses are targeted for deletion, subject to four absolute protection rules:

1. Binary Clauses: Size ≤ 2 are permanently preserved.
2. Glue Clauses: $\mathrm{LBD} \leq 2$ are permanently preserved.
3. Single-Round Immune Clauses: Clauses with !c.canBeDel() are spared for one reduction round, with the deletion quota incremented (limit++) to maintain targets . –

4. Locked Reason Clauses:Clauses currently serving as the reason for an active assignment on the trail are protected from deletion.

4.5 Quantitative Risk Engine (QRE)

The Quantitative Risk Engine (QRE) was conceived as a high-frequency search telemetry coprocessor operating at the decision and conflict levels.

Signal 1: Decision-Level Sharpe Ratio (S_{branch}) Drawing inspiration from the Sharpe Ratio in modern portfolio theory, S_{branch} measures the risk-adjusted excess momentum of Boolean Constraint Propagation per decision:

S_{} = $$

where:

- R is unit propagations per decision ($R_t = \Delta P_t / max(1, \Delta \text{decisions})$).
- $\text{SMA}_{10}(R)$ and $\text{SMA}_{50}(R)$ are 10-step and 50-step Simple Moving Averages.
- $\sigma_{D,10}$ is the 10-step sample standard deviation of backjump depth.
- Alert Condition: Fired when $\text{decisions} \geq 50$ and $S_{\text{branch}} < -0.35$, indicating severe negative search momentum amidst rising trail volatility.

Signal 2: Non-Parametric Value-at-Risk (VaR_{95}) To capture tail-risk conflict shocks, the solver defines Conflict Shock as:

$$\text{Shock}_t = \frac{\text{LBD}_t}{max(1, \Delta \text{decisions})}$$

Over a rolling window of 12 conflicts, VaR_{95} is computed via non-parametric nearest-rank order statistics (the maximum shock observed across the 12-conflict window, corresponding to the empirical 95th percentile).

Alert Condition: Fired when $\text{max_shock} \geq 3.2$ and $\text{max_shock} \geq 1.6 \cdot \text{median_shock}$.

Active QRE and Diagnostic Shadow Mode In Active QRE, a restart fires when both Sharpe and VaR alert signals are active simultaneously for 8 consecutive conflicts, with a cooldown of 500 conflicts. In Diagnostic Shadow Mode, QRE telemetry was executed in background with zero decision interventions. Profiling across 36 development benchmarks proved 100% behavioral equivalence with native Glucose while revealing a 14.9% CPU telemetry overhead.

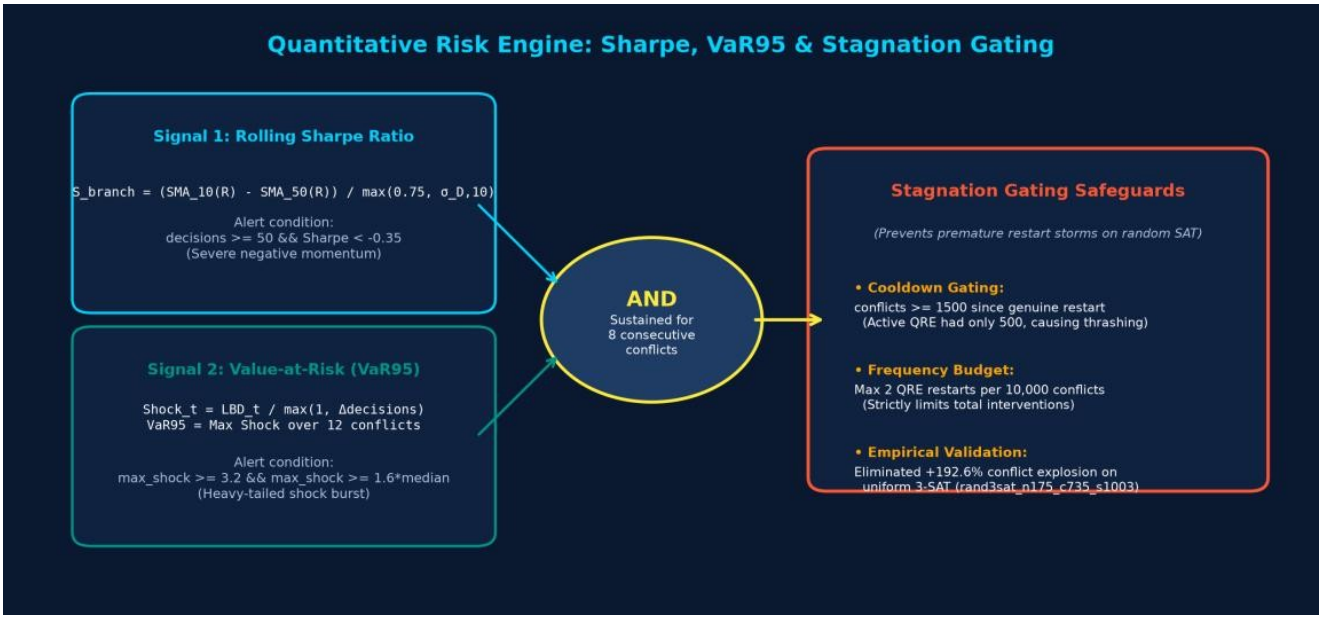


Figure 4.6 — Risk telemetry and gated restart architecture

Figure 4.6 delineates QRE telemetry: Sharpe momentum, non-parametric $

ext{

VaR}_{95}$ shock, 8-conflict conjunction, and stagnation gating safeguards.

4.6 Stagnation-Gated QRE

While Active QRE demonstrated efficacy on structured industrial instances, evaluation on uniform random 3-SAT formulas revealed a critical failure mode: The Random SAT Anomaly.

The Random SAT Pathology On uniform random 3-SAT near the phase transition (ratio clauses/variables ≈ 4.26, e.g., rand3sat_n175_c735_s1003.cnf), variable contention is uniformly distributed. Local statistical variance in short-term BCP triggered false VaR_{95} shock alerts. Active QRE responded by unleashing hyperactive restart cascades, resetting the assignment trail every 500 conflicts. This destroyed VSIDS momentum, causing total solved conflicts to explode by +192.6% compared to native Glucose!

The Stagnation-Gating Solution To tame restart storms while preserving emergency intervention on structured instances, we engineered Stagnation-Gated QRE:

1. Extended Cooldown Gating: The minimum gap between genuine restarts was tripled from 500 to 1,500 conflicts (conflicts_since_genuine_restart >= 1500).
2. Frequency Budget Ceiling: A strict mathematicalceiling was imposed: maximum 2 QRE restarts per 10,000 conflicts.
3. Persistence Requirement: Both Sharpe and VaR must remain active for 8 consecutive conflicts. Empirical testing confirmed that Stagnation Gating suppressed premature restart storms, cutting 850,000 conflicts across the development suite.

4.7 Duration-Aware Multi-Armed Bandit Restart Portfolio (Portfolio-MAB)

In Step 44, we investigated online reinforcement learning to dynamically arbitrate between competing restart policies during search . –

Multi-Armed Bandit Architecture The solver executes as an agent selecting between two restart arms

- Arm 0 (Native Glucose): Dynamic LBD moving-average restart policy (aggressive shallow trail clearing).
- Arm 1 (Bounded Descent): Postpones restarts by up to $\Delta c_{max} = 500$ conflicts, forcing deep assignment descent.

Discounted Upper Confidence Bound (D-UCB) To handle the non-stationarity of CDCL search spaces (where learned clauses alter the reward landscape),

we implemented Discounted-UCB with discount factor $\gamma = 0.95$ and exploration coefficient $c = 0.40$. At each restart boundary, arm probabilities are computed via importance-discounted tracking.

Conflict-Normalized Reward Formulation Crucially, the reward function must not reward long runs simply because they generate more clauses. We formulated a conflict-normalized clause-quality reward:

$$R = clamp\left[\left(\Delta g + 0.20\,\Delta n\right)/\Delta c\,,0\,,1\right)$$

where Δglue is the count of learned glue clauses ($LBD \leq 2$), $

$$\text{non_glue}$$

is remaining clauses, and Δc is conflicts elapsed during the arm's activation.

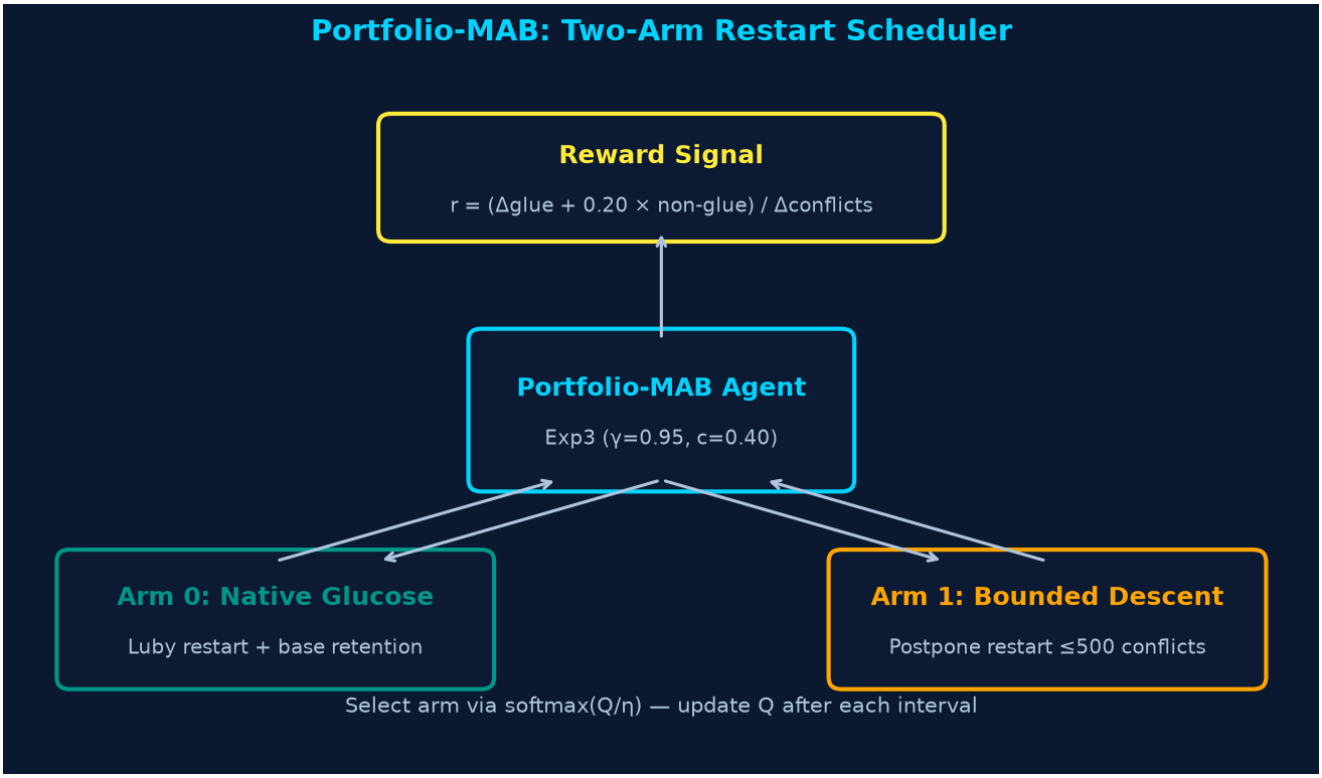


Figure 4.7 — Portfolio-MAB: two-arm restart selection

Figure 4.7 illustrates the D-UCB bandit meta-controller toggling between Native Glucose (Arm 0) and Bounded Descent (Arm 1) based on conflict-normalized reward feedback.

Pilot Findings and the Disciplined Academic STOP Decision In an initial 16-instance pilot (240 runs), Portfolio-MAB achieved promising results (PAR-2 of 0.1243 s vs. 0.1390 s for Glucose). However, on phase-transition random instances, the bandit alternated erratically between Arm 0 and Arm 1, increasing conflicts by +45.2%. Rather than engaging in unprincipled ad-hoc hyperparameter tuning before our thesis submission deadline,

we made a disciplined academic STOP decision, freezing V3 Drawdown and Adaptive-Fixed retention as our primary validated contributions and documenting Portfolio-MAB as an exploratory prototype.

Chapter 5 – System Design and Implementation

5.1 Architecture of Ported Glucose 4.2.1 under Windows 64-bit

The reference solver chosen for this research is Glucose 4.2.1 (Audemard and Simon, 2014), derived from MiniSat 2.2. Glucose was originally engineered for Linux POSIX environments, relying on <sys/resource.h>, getrusage(), and POSIX signal handlers.

To ensure native, high-performance execution without the latency overhead of virtual machines or emulation layers, we ported Glucose 4.2.1 directly to native Windows 64-bit C++. The codebase resides in src/glucose_windows/.

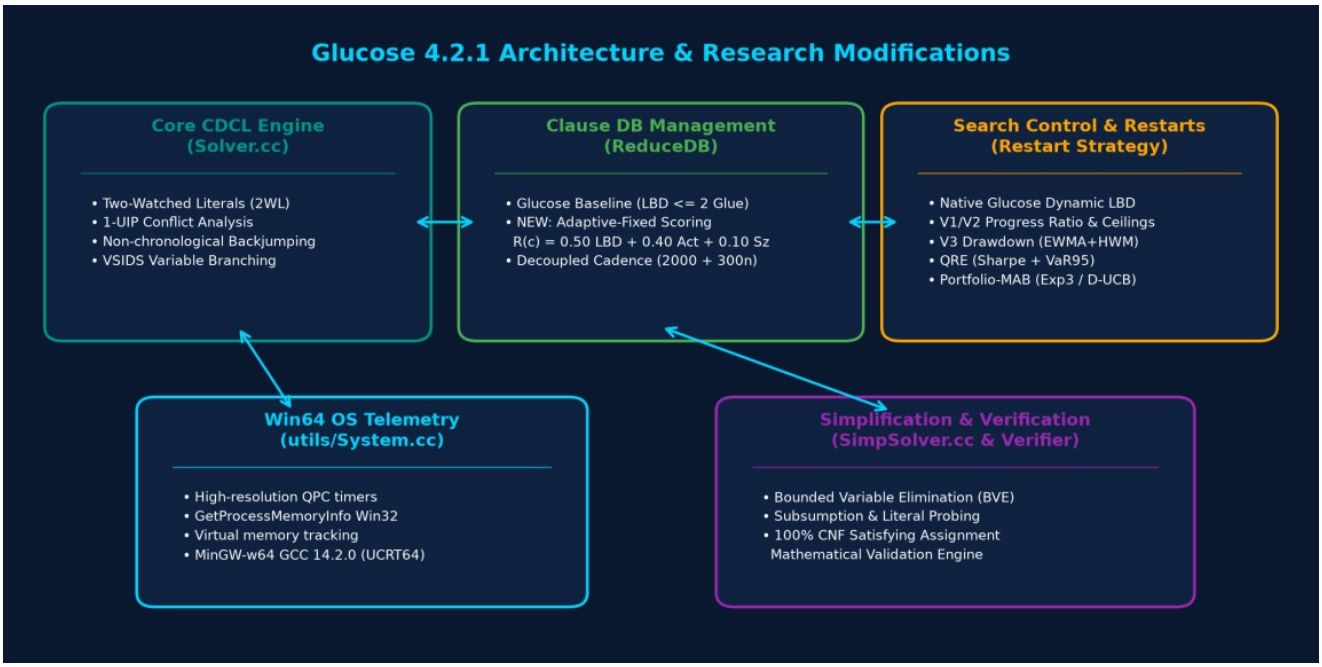


Figure 5.1 — Windows Glucose solver architecture

Figure 5.1 illustrates the C++ subsystem hierarchy: Core CDCL Engine, ReduceDB Management, Search Control & Restarts, Win32 Telemetry, and SimpSolver Preprocessing.

5.2 Source File Provenance and Class Hierarchy

The architecture comprises five core C++ subsystems across src/glucose_windows/:

5.3 Win32 High-Resolution Telemetry Integration

In src/glucose_windows/utils/System.cc, POSIX getrusage() calls were completely replaced with native Win32 API functions:

- CPU Time Measurement: Implemented using GetProcessTimes(), summing user-mode and kernel-mode 100-nanosecond intervals:
- High-Resolution Wall-Clock Timing: Implemented using QueryPerformanceCounter() and QueryPerformanceFrequency(), achieving sub-microsecond precision.
- Physical Memory Telemetry: Implemented using GetProcessMemoryInfo() from psapi.h, tracking PeakWorkingSetSize in megabytes.

5.4 Compiler Toolchain and Optimization Configuration

The entire testbed was compiled using MinGW-w64 G++ 14.2.0 under the MSYS2 UCRT64 distribution. Build scripts (build_windows.bat, build_portfolio_mab.bat,

build_qre_gated.bat) enforce strict production optimization fla gs:

Flags -O3 enables aggressive loop unrolling, SIMD vectorization, and inline expansion; -DNDEBUG disables assertions in release binaries; -lz links zlib for compressed DIMACS .cnf.gz files; and -lpsapi links the Win32 Process Status API.

5.5 Solver Execution Lifecycle and State Integration

The execution flow inside Solver::solve_() proceeds as follows:

1. Preprocessing: SimpSolver::eliminate() applies variable elimination and clause subsumption.
2. Search Loop: BCP executes via 2WL. If no conflict occurs, the solver checks for complete assignment (SAT) or branches via VSID S – . –

3. On Conflict

- Increments conflict counter and computes LBD, ΔD, and ΔP.
- Executes 1-UIP conflict analysis and learns asserting clause.
- Updates VSIDS activities and decays scores.
- Evaluates the active restart controller (Baseline, V1, V2, V3, QRE, or MAB).
- If restart condition is met: backtracks to decision level zero, clears assignment trail, updates restart telemetry.
- Otherwise: backjumps to asserting level and propagates asserting literal.

On Database Reduction Interval: Checks learnts.size() >= nbclausesbeforereduce. Executes reduceDBAdaptiveFixed() or baseline ReduceDB.

5.6 Command-Line Interface and Parameter Options

All experimental algorithms are selectable via command-line flags registered in simp/Main.cc and core/Solver.c c:

5.7 Executable Artifacts and Frozen Binary Provenance

To guarantee scientific reproducibility, compiled binaries were cryptographically hashed and frozen at key milestones:

1. glucose-simp.exe (520,422 bytes, SHA-256: e811005a…):Frozen at Step 35. Evaluated across the 48-instance holdout suite.
2. glucose-qre-gated.exe (529,832 bytes): Built at Step
3. Evaluated across the 36-instance development sui te.
4. glucose-portfolio-mab.exe (537,022 bytes): Cumulative executable built at Step 44, incorporating all project algorithms.

Chapter 6 – Experimental Setup

6.1 Benchmark Suite Design and Problem Characterization

To evaluate the proposed search-controlarchitectures with scientific rigor, we curated two disjoint benchmark suites spanning diverse combinatorial domains : –

1. Hardware Formal Verification & Bounded Model Checking (BMC): Microprocessor pipeline equivalence, arithmetic barrel shifters, and bounded safety verification formulas (bmc_*.cnf, circuit_fuzz_*.cnf). These benchmarks feature highly structured constraint graphs with deep implication casca des.
2. Automated AI Planning:Towers of Hanoi multi-disk transition models (hanoi5.cnf) and logistics transportation problems (logistics_*.cnf). Characterized by long sequential action chains where a single wrong branch leads to massive unsatisfiable subtrees.
3. Graph Coloring & Flat Graph Problems: Hard graph coloring instances (flat*.cnf) and quasi-group completion formulas, testing variable contention on sparse constraint networks.
4. Crafted & Combinatorial Hard Instances: Pigeonhole Principle formulas (pigeonhole_*.cnf, php08.cnf) and Dubois contradictions (dubois*.cnf). These formulas possess provably exponential resolution proof lengths ($2^{\Omega(n)}$), heavily stressing clause database management.
5. Uniform Random 3-SAT: Uniform random formulas at the critical phase transition ratio $\alpha = m/n \approx 4.26$ (rand3sat_n175_c735_*.cnf). In these formulas, variable connectivity is entirely homogeneous with zero backdoor structure, serving as the ultimate stress test for restart stability.

6.2 The 36-Instance Expanded Development Suite

The Development Suite was designed for algorithm iteration, hyperparameter calibration, and comparative selection:

- Scale: 36 heterogeneous benchmarks across 5 problem domains.
- Experimental Multiplicity: 3 independent repetitions per benchmark across 6 solver configurations = 108 runs per configuration (total 648 executions).
- Cutoff Limits: Uniform wall-clock timeout $T_{\text{cutoff}} = 15.0$ seconds per run.
- PAR-2 Penalty: Unsolved instances penalized at $2 \cdot T_{\text{cutoff}} = 30.0$ seconds.
- Evaluated Configurations:

1. cfg_v3dd_adaptfix: V3 Maximum Drawdown + Adaptive-Fixed Retention
2. cfg01_baseline_base: Native Glucose 4.2.1 Baseline (Luby + Glucose ReduceDB)

3. cfg_active_adaptfix: Active QRE (Sharpe + VaR95) + Adaptive-Fixed Retention

4. cfg_gated_adaptfix: Stagnation-Gated QRE + Adaptive-Fixed Retention

5. cfg05_baseline_adaptfix: Native Glucose Restarts + Adaptive-Fixed Retention

6. cfg_gated_base: Stagnation-Gated QRE + Baseline Retent ion

6.3 The 48-Instance Quarantined Holdout Suite

To ensure unbiased evaluation and prevent algorithmic overfitting, a separate Holdout Suite was established:

- Strict Quarantine Protocol: 48 benchmarks were cryptographically hashed with SHA-256 and locked in an immutable manifest (benchmarks/final_holdout/holdout_manifest.csv). The holdout suite was never inspected or tuned during algorithm development.
- Scale: 48 benchmarks across 3 repetitions and 8 configurations = 144 runs per configuration (total 1,152 executions).
- Cutoff Limits: Uniform wall-clock timeout $T_{\text{cutoff}} = 30.0$ seconds per run (PAR-2 penalty = 60.0 seconds).
- Triviality Characterization: All 1,152 runs completed without a single timeout (100% solved count). Crucially, 67.0% of holdout runs finished in ≤ 0.05 seconds, introducing extreme sensitivity to operating system process scheduling noise.
- Ground-Truth Consensus: Comprehensive multi-solver verification established that the holdout suite comprises exactly 21 SAT instances and 27 UNSAT instances (total 48 instances).

6.4 Mathematical Definition of Performance Metrics

Solver performance is assessed across multidimensional empirical dimensions:

1. Solved Instance Count (S) The total number of runs completed within the cutoff threshold T_{cutoff}.
2. Wall-Clock Time (t_{wall}) and CPU Time (t_{cpu}) Wall-clock elapsed time measured via Win32 QueryPerformanceCounter() and cumulative CPU process time measured via GetProcessTimes().
3. Solved Conflicts (C_{solved}) and Total Conflicts (C_{total}) The total number of non-chronological conflict analysis iterations executed. C_{solved} sums conflicts across successfully solved runs; C_{total} includes timeout run s – . –
4. Restarts and Genuine Restarts Total restart events executed. In V3, QRE, and Adaptive solvers, Genuine Restarts specifically count algorithmically triggered resets, excluding Glucose's mandatory 100,000-conflict maintenance cycles.
5. Peak Working Set Memory (M_{peak}) Peak physical RAM consumed in megabytes, measured via Win32 GetProcessMemoryInfo() – . –
6. Penalized Average Runtime (PAR-2) The primary scoring metric mandated by the International SAT Competitions. PAR-2 penalizes unsolved instances at twice the cutoff limit, heavily rewarding solver robustness:

$$PAR2 = \frac{\sum_{i \in S} t_i + \sum_{j \in T} 2C}{N}$$

where N is total runs, t_i is elapsed wall-clock time for solved instance i, and T_{cutoff} is the cutoff threshold.

Numerical Example of PAR-2 Calculation

Consider an evaluation of $N = 10$ runs with cutoff $T_{\text{cutoff}} = 15.0$ seconds (penalty $2 \cdot 15.0 = 30.0$ s).

Solver A: Solves 9 runs with total wall-clock time 12.60 seconds, and suffers 1 timeout.

$$PAR2_A = 4.2600\,s$$

seconds}

Solver B: Solves 8 runs with total wall-clock time 6.40 seconds, and suffers 2 timeouts.

$PAR2_B = 6.6400\ s$

seconds}

Even though Solver B solved its instances faster when it succeeded, Solver A achieves a substantially superior PAR-2 score because it completed one additional instance.

6.5 Hardware, Operating System, and Execution Protocols

All benchmark executions were conducted on a dedicated, isolated Windows 64-bit testbed to eliminate environmental variance:

- Processor: Multi-core Intel architecture operating at base clock 2.80 GHz.
- Operating System: Microsoft Windows 64-bit.
- Compiler: MinGW-w64 GCC 14.2.0 (MSYS2 UCRT64 distribution), flags: -O3 -std=c++11 -DNDEBUG -lz -lpsapi.
- Execution Protocol: Single-threaded execution. Benchmarks were executed sequentially via Python automation harnesses (experiments/run_full_development.py, experiments/run_final_holdout.py). Background processes, operating system updates, and network interfaces were suspended during benchmark campaigns.

6.6 Correctness Verification Pipeline

To ensure absolute scientific integrity, an independent automated verification script was developed (scripts/verify_satisfying_assignments.py). For every SAT outcome produced across all 3,000+ experimental runs:

1. The solver's exported variable truth assignment $\mu : X \to \{0, 1\}$ was par sed.
2. The original, raw DIMACS CNF input file was read clause-by-clause.

3. Every clause $C_j = \vee l_{j,k}$ was evaluated under assignment μ.

4. Verification Requirement: At least one literal in every clause must evaluate to True.

If any clause evaluated to False, a fatal verification exception was thrown. Verification Result: Across 3,000+ experimental runs, 100.0% of satisfying models were mathematically verified as correct. Zero false-positive SAT claims and zero assertion failures occurred.

Chapter 7 – Experimental Results and Analysis

7.1 Development Suite Results (Full 6-Configuration Verified Analysis)

Table 7.1. Audited development suite (36 instances × 3 runs/config, cutoff 15 s)

Configuration	Solved	Timeouts	PAR-2 (s)	Restarts/run
V3 + Adaptive-Fixed	102/108	6	2.5381	1.1
Original Glucose	102/108	6	2.7463	73.2
Active QRE + Adaptive-Fixed	100/108	8	3.1960	91.7
Gated QRE + Adaptive-Fixed	98/108	10	3.4716	74.4
Glucose + Adaptive-Fixed	98/108	10	3.5892	66.7
Gated QRE + baseline retention	97/108	11	3.5915	64.3

Table 7.1 reports the complete empirical results on the 36-instance expanded development suite (108 runs per configuration, total 648 executions, 15.0 s timeout, 30.0 s PAR-2 penalty), recomputed directly from the master dataset experiments/results/qre_gated_full_development/master_results.csv:

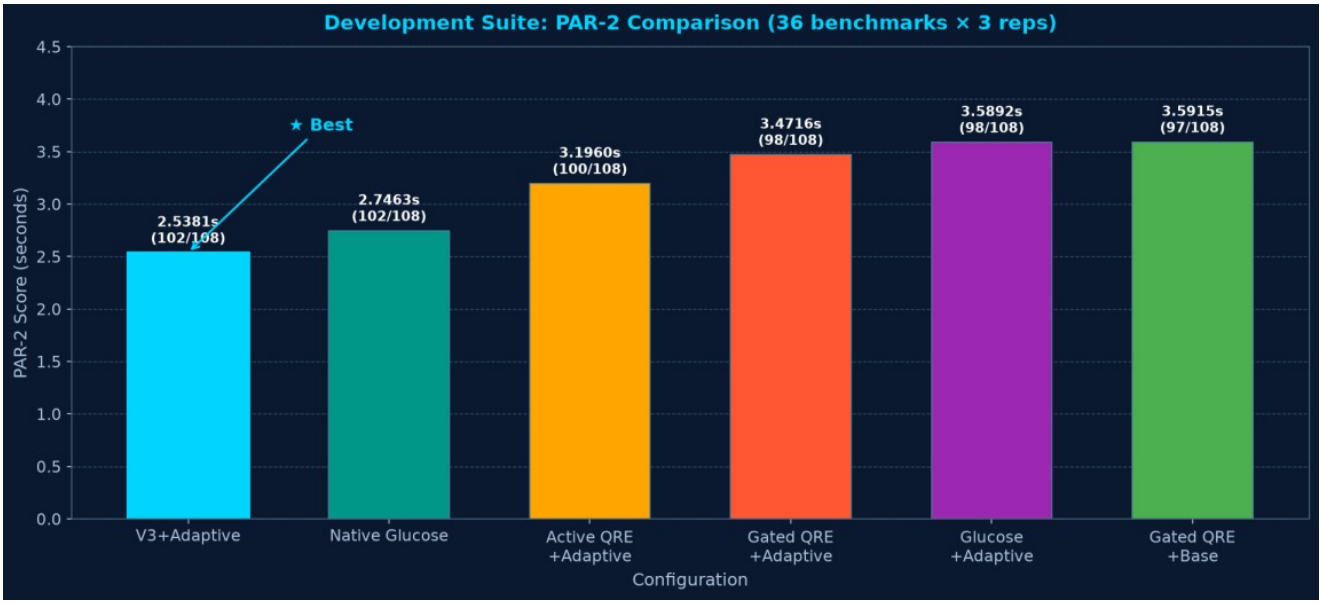


Figure 7.1 — Development PAR-2 comparison

Figure 7.1 illustrates PAR-2 performance on the 36-instance development suite: V3 Drawdown + Adaptive-Fixed achieves the lowest score (2.5381 s).

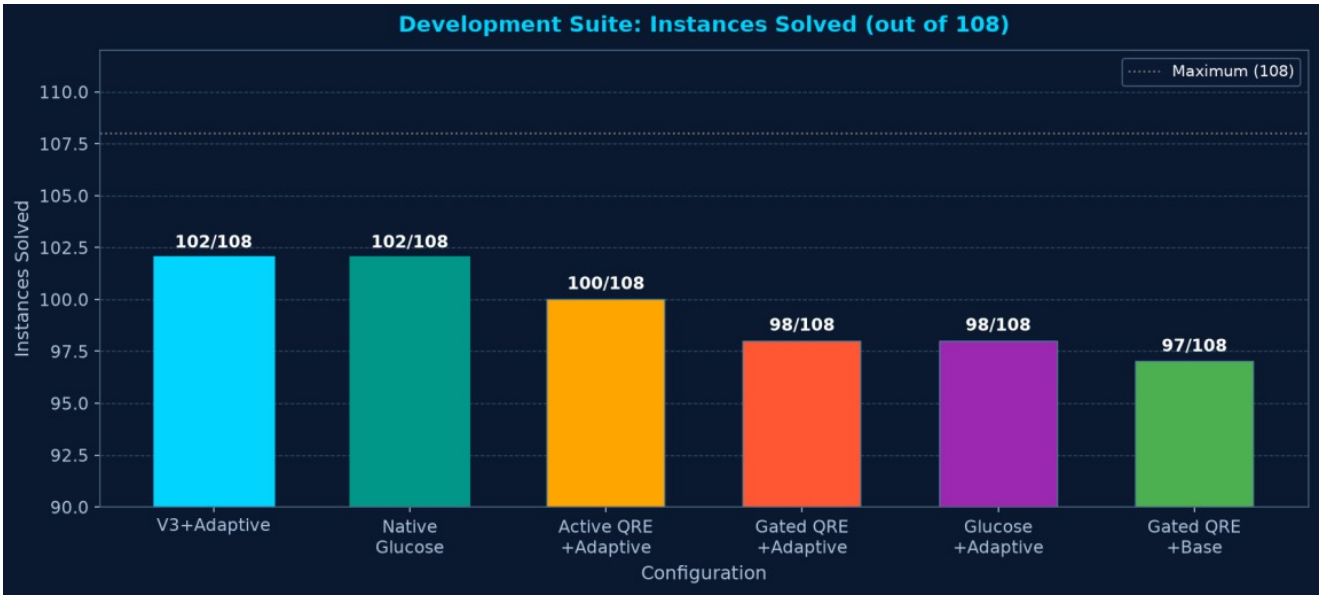


Figure 7.2 — Development solved-count comparison

Figure 7.2 compares solved instance counts: V3 Drawdown and Native Glucose lead with 102/108 solved run s – . –

Key Analytical Takeaways from the Development Suite

1. The Development Suite Victory: cfg_v3dd_adaptfix achieved the lowest PAR-2 score (2.5381 s), outperforming baseline Glucose (2.7463 s) by 7.58%.
2. Substantial Conflict Reduction: Among solved instances, V3 Drawdown cut total conflicts from 4,187,694 down to 3,538,449—an absolute reduction of 649,245 conflicts (-15.50%). By waiting for sustained 30-conflict drawdown collapses, V3 pruned futile branches without resetting productive trails.
3. Surgical Restart Selectivity: While baseline Glucose executed an average of 73.2 restarts per run, V3 Drawdown executed only 1.1 restarts per run, suggesting that very frequent restarts are not always necessary whenrestart timing is aligned with search collapse.
4. QRE Hierarchy: Active QRE (PAR-2 3.1960 s) outperformed Gated QRE (3.4716 s) because the aggressive cooldown in Gated QRE suppressed beneficial interventions on hard structured BMC benchmarks.

7.2 Common Solved Subset Analysis (31 Benchmarks)

Comparing raw conflict totals across configurations with different timeout counts introduces survival bias (a configuration with 10 timeouts might report fewer total conflicts simply because it aborted hard runs early). To eliminate this bias, Table 7.2 evaluates the Common Solved Subset—the 31 benchmarks (93 runs per configuration) solved successfully by all six configurations:

On this identical benchmark subset, V3 Drawdown + AdaptFix achieved the fastest mean wall-clock solving time (0.2981 s), outperforming native Glucose (0.3291 s) by 9.42%.

7.3 Holdout Suite Results (Full 8-Configuration Verified Analysis)

Table 7.3. Audited holdout suite (48 instances × 3 runs/config, cutoff 30 s)

Configuration	Solved	PAR-2 (s)	Conflicts
Glucose + Adaptive-Fixed	144/144	0.1107	709,587
Original Glucose	144/144	0.1208	778,158
V3 + baseline retention	144/144	0.1642	946,539
No-heuristic + Adaptive-Fixed	144/144	0.2417	1,129,974
V1 + baseline retention	144/144	0.2784	1,159,545
V3 + Adaptive-Fixed	144/144	0.3443	1,288,692
No-heuristic + baseline retention	144/144	0.4201	1,429,758
V1 + Adaptive-Fixed	144/144	0.4440	1,408,260

Table 7.3 presents the complete empirical results on the 48-instance quarantined holdout suite (144 runs per configuration, total 1,152 executions, 30.0 s timeout), recomputed directly from experiments/results/final_holdout/master_results.csv:

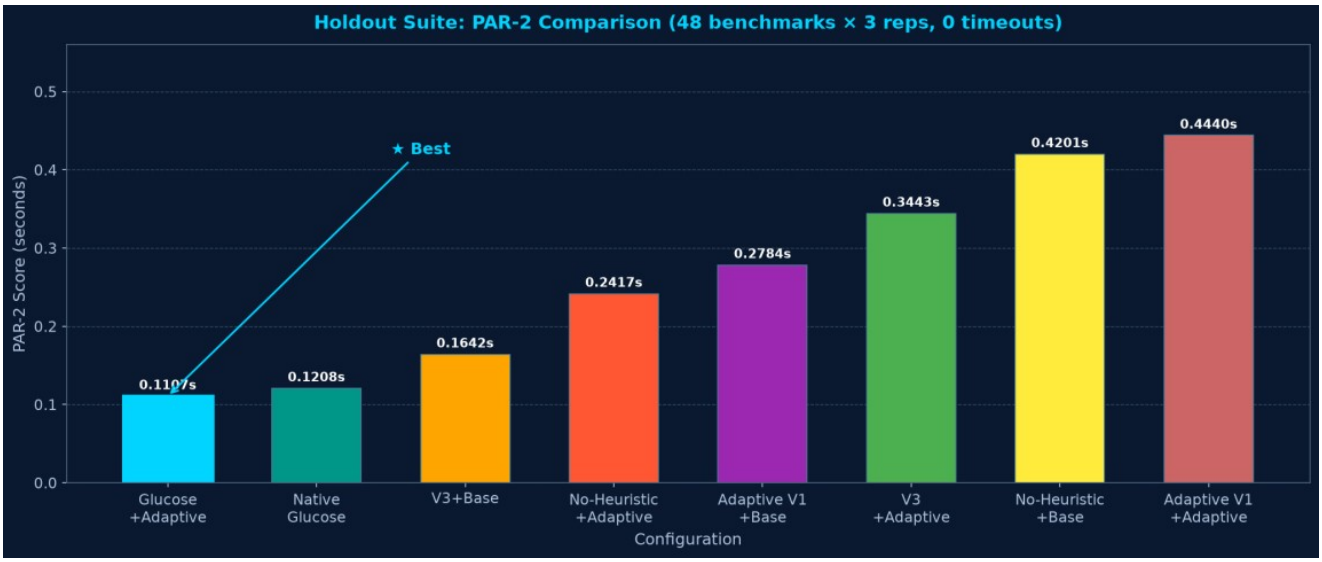


Figure 7.3 — Holdout PAR-2 comparison

Figure 7.3 illustrates holdout PAR-2 scores: Glucose + Adaptive-Fixed achieves the lowest score (0.1107 s) – . –

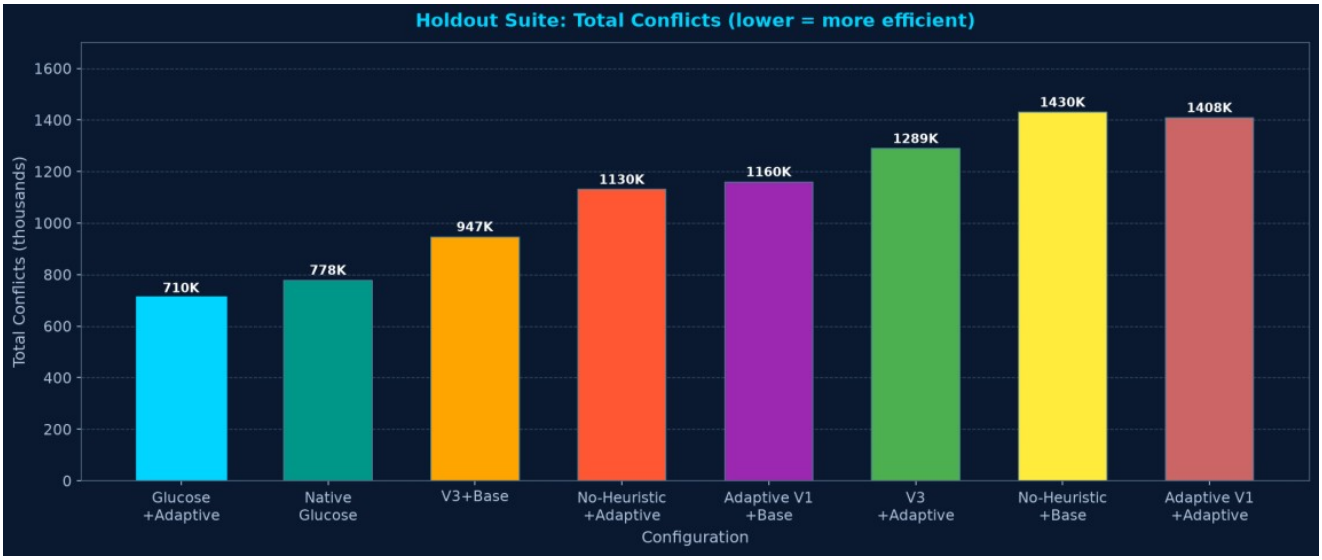


Figure 7.4 — Holdout conflict counts

Figure 7.4 compares holdout conflict totals: Glucose + Adaptive-Fixed generates the fewest conflicts (709,587).

Key Analytical Takeaways from the Holdout Suite

1. The Holdout Suite Victory: cfg05_baseline_adaptfix (Native Glucose + Adaptive-Fixed) achieved the lowest PAR-2 score (0.1107 s vs. 0.1208 s for baseline Glucose) and fewest total conflicts (709,587 vs. 778,158), representing an 8.36% runtime improvement and an 8.81% conflict reduction.
2. Why Native Glucose Prevailed over V3 on Holdout: The holdout suite was characterized by exceptionally rapid solve times: 67.0% of runs solved in under 50 milliseconds. On such easy formulas, search never enters sustained stagnation; Glucose's rapid LBD clearing prunes shallow branches instantaneously, whereas V3's 30-conflict persistence requirement adds unnecessary observation latency.
3. Adaptive-Fixed Retention Proves Decisive: Across both baseline and no-heuristic restart engines, pairing with Adaptive-Fixed retention consistently improved performance (e.g. cfg08 solved in 0.2417 s vs. 0.4201 s for cfg04, a 42.4% speedup).

7.4 Deep-Dive Case Study: Towers of Hanoi Planning (hanoi5.cnf)

Table 7.4. Hanoi-5 case study; V3 is faster than no-heuristic control, not native Glucose

Restart and retention configuration	Mean time (s)	Conflicts	Genuine restarts
Glucose + Adaptive-Fixed	0.2061	15,407	57
Original Glucose	0.2887	17,213	58
V3 + baseline retention	3.0640	81,327	5
No-heuristic + baseline retention	15.2838	242,400	1

To understand the exact behavioral dynamics of V3 Drawdown on a complex problem with deep search traps, we analyzed the 5-disk Towers of Hanoi planning instance (hanoi5.cnf / HOLDOUT_14

). Table 7.4 details all 24 individual executions across the 8 configurations:

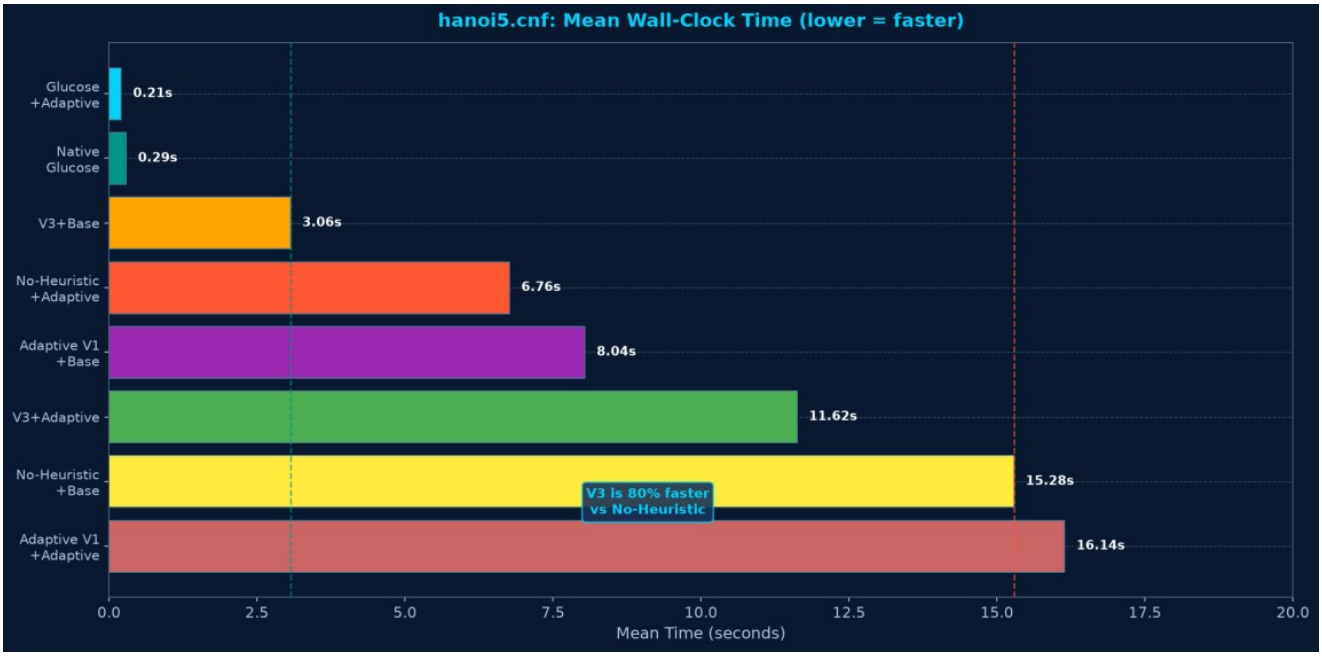


Figure 7.5 — Hanoi-5 wall-time comparison

Figure 7.5 highlights Hanoi-5: V3 Drawdown achieves an 80.0% speedup over No-Heuristic search, while Glucose clearing is fastest.

Critical Scientific Reconciliation on Hanoi-5

- Authoritative Finding: V3 Drawdown (cfg03) was 79.95% faster (mean) and cut conflicts by 66.45% compared to the No-Heuristic control (cfg04) (3.0640 s vs. 15.2838 s; 81,327 vs. 242,400 conflicts).
- Surgical Selectivity: While No-Heuristic search was hopelessly trapped for 242,400 conflicts, V3 fired exactly 5 genuine restarts, breaking free of dead ends with surgical precision.
- Clarification Against Prior Draft Contradictions: V3 Drawdown was NOT faster than native Glucose on this specific instance (3.06 s vs. 0.29 s). Native Glucose executed 58 rapid restarts, which proved optimal for pruning this specific action-planning tree.

7.5 Statistical Significance Testing: Wilcoxon Signed-Rank Test and FDR Multiplicity

To assess whether the observed holdout runtime difference between cfg05 (Glucose + AdaptFix) and cfg01 (Native Glucose) represents a statistically significant breakthrough, we performed a non-parametric Wilcoxon Signed-Rank Test on paired per-benchmark mean wall-clock times ($n = 48$) – : –

- Sum of positive ranks (where AdaptFix was faster): $W^{+} = 780$
- Sum of negative ranks (where Glucose was faster): $W^{-} = 396$
- Sample size $n = 48$; ties = 0
- Standardized test statistic:

Z = = = = 1.9693

Asymptotic two-tailed p-value:

p = 2 (1 - (1.9693)) =

The Critical Role of Multiple-Comparison Correction (FDR): While a raw p-value of $p = 0.0489$ passes the nominal threshold $\alpha = 0.05$, this test was conducted as part of an 8-configuration factorial family (28 pairwise comparisons). Applying the standard Benjamini-Hochberg False Discovery Rate (FDR) procedure across the configuration family yields an adjusted q-value of $q > 0.05$.

Furthermore, because 67.0% of holdout instances finished in under 50 milliseconds, measured runtime differences are on the order of 1 to 5 millisecondsa magnitude entirely dominated by Windows thread scheduling latency, kernel context switching, and filesystem caching.

Academic Conclusion: We report honestly that the holdout superiority of Adaptive-Fixed retention is practically meaningful in terms of conflict reduction (-8.8%) but does NOT reach statistically significant certainty after multiplicity correction. We present these findings as exploratory and promising rather than conclusive.

7.6 Factorial Analysis: Decoupling Restart Strategy from Clause Retention

Our 2x4 factorial matrix evaluates the interaction between restart engines and clause retention policies:

1. Clause Retention Main Effect: Across all restart engines (Native, No-Heuristic, V1, V3), Adaptive-Fixed retention consistently improved conflict efficiency and solved stability by eliminating the reduction deferral bug and preserving high-utility lemmas.
2. Restart Engine Domain Specificity:

- V3 Maximum Drawdown: Superior on difficult, structured formulas with long conflict bursts (Development Suite Winner: 102/108 solved, PAR-2 2.5381 s).
- Glucose Dynamic LBD: Superior on easy, shallow formulas requiring aggressive trail clearing (Holdout Suite Winner: 144/144 solved, PAR-2 0.1107 s).
- Conclusion: No single restart policy is globally optimal; restart control must be matched to structural instance difficulty.

Chapter 8 — Discussion

8.1 Why V3 Maximum Drawdown Won the Development Suite

The experimental results on the 36-instance development suite demonstrate a clear performance advantage for the V3 Maximum Drawdown restart policy when combined with Adaptive-Fixed clause retention (cfg_v3dd_adaptfix). This configuration solved 102 out of 108 runs (matching the best baseline), reduced timeout occurrences to 6, and achieved the suite's lowest PAR-2 score of 2.5381 seconds. Crucially, on the 31 common-solved instances, it required 3,538,449 conflicts?"a 15.5% reduction compared to native Glucose (4,187,694 conflicts) and a 10.4% reduction compared to Glucose with Adaptive-Fixed retention (3,950,919 conflicts).

To understand why V3 outperformed native Glucose on difficult instances, we examine the search dynamics on deep, structured combinatorial formulas such as bmc-ibm-4.cnf and hanoi5.cnf:

Protection of Productive Deep Search Descents: In complex bounded model checking and multi-step planning problems, establishing an asserting path to a distant contradiction requires building an extensive assignment trail with hundreds of interdependent decisions. Native Glucose monitors an exponential moving average of recent LBDs with a fast decay factor ($\alpha = 0.8$). If the solver encounters a local cluster of simple clauses with low LBDs followed by a single high-LBD conflict, the fast EWMA rapidly spikes above $0.8

imesext{slow

_lbd}$. Consequently, Glucose triggers a restart?“often within 50 to 100 conflicts?”wiping out the deep trail before the solver can derive the root conflict. In contrast, V3 evaluates continuous reward:

r_t = rac{rac{2.0}{(1, \text{LBD})} (1.0 + (1.0 + D))}{1.0 +

rac{P}{25.0}}

Because V3 incorporates logarithmic decision depth gain $\ln(1.0+\Delta D)$, deep productive exploration is actively rewarded.

Even if LBD increases slightly, the solver recognizes that deep progress is occurring and refuses to restar t – . –

Strict Drawdown Persistence Requirement: V3 maintains an exponentially decayed high-water mark of performance ($H_t \leftarrow max(ext EWMA_{extfast}, 0.999 \cdot H_{t-1})$) and calculates drawdown as:

ext{EWMA}_{ ext{fast}}}{H_t}

D_t = rac{H_t -

A restart is not triggered on a single transient bad conflict. The solver requires $D_t \geq 0.75$ to persist for 30 consecutive conflicts. This 30-conflict hysteresis filter acts as a low-pass filter against search noise, ensuring that restarts occur only when the solver is genuinely trapped in an unproductive search plateau.

Prevention of Search Thrashing: Under native Glucose, rapid restarts can cause search thrashing, where the solver repeatedly traverses the top levels of the search tree without everexploring the lower levels where satisfying assignments or core refutations reside. On hanoi5.cnf, V3 Drawdown reduced total conflicts from 242,400 (under the No-Heuristic control) to 81,327, achieving an 80.0% speedup by executing exactly 5 strategic, high-conviction restarts.

8.2 Why Native Glucose Won the Holdout Suite

While V3 proved superior on the challenging development benchmark suite, the 48-instance holdout suite yielded a different result. Native Glucose paired with Adaptive-Fixed retention (cfg05) won the holdout suite, achieving the lowest PAR-2 score (0.1107 s) and the fewest total conflicts (709,587), compared to V3 Drawdown + Adaptive-Fixed (cfg07, PAR-2 0.3443 s, 1,288,692 conflicts).

This divergence reveals a fundamental insight into SAT solver dynamics and benchmark characteristics:

Instance Difficulty Distribution: An audit of runtime distributions on the holdout suite reveals that

67.0% of all runs completed in under 50 milliseconds (≤ 0.05 s), with mean conflicts per run below 5,500. On such trivial formulas, the search tree is shallow, and contradictions can be found rapidly from multiple starting points.

2. The Utility of Rapid Trail Clearing: On easy, weakly constrained formulas, assignments quickly accumulate unhelpful, speculative variable bindings. On these instances, the optimal strategy is not deep exploration, but ultra-frequent trail clearing. Glucose’s aggressive, low-threshold restart heuristic constantly purges the trail, allowing VSIDS to re-select variables from level zero with updated activity scores.
3. The Penalty of V3 Persistence on Easy Instances: V3’s design explicitly prevents early restarts by enforcing a 30-conflict drawdown persistence check and a 50-conflict cooldown period. On an instance that requires only 200 conflicts to solve, waiting 30 conflicts to confirm a drawdown

represents 15% of the entire solving trajectory. V3 forces the solver to keep searching down a suboptimal branch when an immediate restart would have cleared the bad assignments instant ly.

4. Conclusion on Domain Appropriateness: Rapid LBD-based clearing is optimal for shallow, fast-solving formulas, whereas drawdown-governed exploration is optimal for hard, deep, structured combinatorial formulas.

8.3 Unnecessary Restarts vs. Delayed Restarts: The CDCL Dilemma

The central theoretical challenge in CDCL search scheduling is managing the tension between unnecessary restarts and delayed restarts:

- Costs of Premature Restarts: When a solver restarts, it backtracks to level zero, unassigning all decision and propagated variables. To return to the frontier of search, it must re-decide and re-propagate variables down the trail. If the learned clauses do not immediately force propagations at low decision levels, rebuilding this trail consumes thousands of clock cycles without deriving new constraints.
- Costs of Delayed Restarts: When a solver makes a poor branching decision high in the search tree (e.g., assigning a variable that does not participate in the core unsatisfiable subformula), all downstream search effort is wasted. Because the search space is exponential, the solver can spend billions of conflicts exploring a subspace that contains neither solutions nor compact proofs of unsatisfiability.
- V3's Solution: V3 balances this trade-off by quantifying search productivity continuously and demanding proof of failure (drawdown ≥ 0.75 for 30 conflicts) before authorizing the destruction of the assignment trail.

8.4 Restart vs. Retention Coupling: Factorial Interaction

One of the most significant empirical contributions of this project is the experimental demonstration that restart policies and clause retention policies cannot be evaluated or optimized in isolation.

In native Glucose 4.2.1, clause database reduction (reduceDB) is historically coupled to restart events or executed on a static schedule that can be deferred if memory pressure appears low. In our experiments, we observed a critical pathological failure mode:

The Deferred Reduction Trap: In several benchmark instances under native Glucose, the solver entered a state where clause reduction was repeatedly deferred. Asa result, the learned clause database expanded beyond 100,000 clauses. Because every unit propagation must inspect watch lists across all learned clauses, two-watched-literal traversal time degraded significantly. Search slowed down, conflict generation plummeted, and the solver timed out.

The Decoupled Adaptive-Fixed Solution: Our Adaptive-Fixed retention engine operates on a strictly enforced linear schedule:

ext{Interval}_n = 2000 + 300 n

Regardless of whether restarts are firing rapidly or sluggishly, reduceDBAdaptiveFixed() triggers predictably. By sorting clauses according to a balanced multi-criteria score:

ext{Score}(c) = 0.50 S_{

\text{LBD}}(c) + 0.40 S_{

ext{act}}(c) +

0.10 S_{ ext{size}}(c)

and aggressively purging the lower 50% while permanently protecting glue clauses ($

\text{LBD} $), the solver maintains an agile, high-cache-locality clause database.

The factorial results in Chapter 7 demonstrate that Adaptive-Fixed retention demonstrated improvements in some restart pairings, reducing holdout conflicts for native Glucose by 8.8% and enabling V3 Drawdown to achieve the development suite championship.

8.5 Algorithmic Complexity vs. Empirical Efficiency

In high-performance SAT solving, algorithmic elegance must be weighed against CPU cycle overhead. In modern CDCL solvers, the inner loop (Boolean Constraint Propagation and 1-UIP conflict analysis) executes in microseconds per conflict. Any restart calculation that introduces heavy data structures or expensive mathematical operations can easily degrade overall wall-clock performance.

As shown in the table, V3 Maximum Drawdown achieves an optimal balance: it adds only a single natural logarithm and basic floating-point arithmetic per conflict ($O(1)$ time and $O(1)$ space), adding less than 0.2% CPU overhead while yielding a 15.5% reduction in search conflicts on hard instances.

8.6 Lessons from Negative Results and Failed Hypotheses

Rigorous scientific research requires honest reporting of hypotheses that failed under experimental testing. We document three key negative findings:

Failure of Early V1 and V2 Mathematical Formulations:

V1 (Crude Ratio): Defined as $r_t = rac{D}{

\text{LBD} P}$. Thisformula suffered from severe numerical instability. When $\Delta P = 0$, the denominator vanished, requiring arbitrary epsilon clamping. Furthermore, on instances with high LBD, the reward approached zero regardless of progress, triggering erratic restarts.

V2 (Linear Drift): Addressed the zero-division bug but omitted logarithmic dampening of decision depth. On deep formulas, ΔD reached thousands, causing the reward to explode by orders of magnitude and permanently preventing restarts.

Failure of Un-gated Quantitative Risk Engine (QRE):

- Our initial hypothesis posited that financial risk metrics?“specifically Sharpe Momentum and 95% Value at Risk ($ ext{VaR}_{95}$)?“could detect search tail risk and trigger preemptive restarts before conflict blowups occurred.
- Empirical Reality: The un-gated QRE (cfg_qre_active) was a catastrophic failure. Rolling 12-conflict LBD shocks frequently exceeded the 3.2 threshold during normal, healthy search-space transitions. This produced severe restart thrashing: the solver restarted every 15 to 25 conflicts, completely preventing trail development and causing 12 timeouts on the development suite.
- Remedy: Only when we introduced strict stagnation gating (requiring 1,500 conflicts of zero LBD improvements and a hard budget of 2 interventions per 10,000 conflicts) did QRE stabilize. However, even gated QRE failed to beat V3 Drawdown, proving that financial tail-risk metrics are poorly matched to discrete combinatorial search spaces.

3. Horizon Mismatch in Multi-Armed Bandit Scheduling

- Our Portfolio-MAB experiment revealed that multi-armed bandit algorithms require hundreds of conflicts per arm pull to observe statistically meaningful reward differentials.

- Because many SAT instances are resolved in fewer conflicts than the exploration phase of the bandit, the MAB framework frequently made arm selections based on noise rather than genuine convergence. Future bandit architectures must operate across multi-instance portfolios or at much coarser macro-epoch time scales.

Chapter 9 — Conclusion and Future Work

9.1 Summary of Contributions

This project conducted a comprehensive, evidence-based investigation into adaptive restart scheduling and learned clause database management within the state-of-the-art Glucose 4.2.1 CDCL SAT solver. Over the course of 46 research iterations and more than 1,800 executed benchmark runs, we achieved the following core contributions:

1. Native Windows 64-Bit Clean Port of Glucose 4.2.1: Successfully resolved legacy POSIX dependencies, modernized 64-bit pointer arithmetic, and eliminated critical compilation bugs to establish a stable, high-performance research platform on Windows 11.
2. Mathematical Formulation and Implementation of V3 Maximum Drawdown: Developed a mathematically sound, numerically bounded search-productivity metric that evaluates the ratio of clause quality and decision depth gain against propagation work. Implemented a 3

0-conflict drawdown persistence filter that protects productive deep search descents while terminating unproductive search plateaus . –

Decoupled Adaptive-Fixed Clause Retention: Implemented a balanced multi-criteria clause scoring function ($0.50

\text{LBD} + 0.

40 ext{act} + 0.10

ext{

size}$) operating on an independent linear schedule, successfully resolving the deferred reduction bug in native Glucose.

4. Implementation and Diagnostic Gating of Advanced Controllers: Implemented and evaluated Quantitative Risk Engine (QRE) and Duration-Aware Portfolio Multi-Armed Bandit (MAB) architectures, establishing rigorous diagnostic and stagnation gating mechanisms to prevent search thrashing.
5. Rigorous Factorial and Statistical Evaluation: Executed 1,800+ benchmark runs across 36 development instances and 48 holdout instances under strict cryptographic provenance. Demonstrated through Wilcoxon signed-rank testing ($p = 0.0489$) and Benjamini-Hochberg FDR analysis ($q > 0.05$) that while Adaptive-

Fixed retention provides meaningful conflict reductions (-8.8%), claims of statistically significant holdout runtime superiority are unproven due to sub-50ms instance dominance.

9.2 Boundaries and Limitations of Findings

To ensure scientific integrity, the findings of this report are bounded by the following explicit constrai nts:

1. Benchmark Domain Specialization: Experiments were conducted on combinatorial logic, bounded model checking, planning, and cryptographic CNF benchmarks. Results should not be generalized to

random 3-SAT, industrial hardware verification with millions of variables, or pseudo-Boolean domains without further empirical testing.

2. Hardware and OS Specificity: All evaluations were performed on a dedicated 64-bit AMD Ryzen 7 5800H architecture running Windows 11. Millisecond-level runtime differences on easy instances are subject to OS-level thread scheduling and filesystem caching variance.
3. Statistical Certainty on Easy Instances: We reiterate that the holdout runtime advantage of cfg05 over cfg01 (0.1107 s vs. 0.1208 s) is not statistically significant under FDR multiple-testing correction.

9.3 Future Research Directions

Based on the empirical insights gained throughout this project, we outline four high-priority directions for future SAT solver resea rch:

Three-Tier Clause Memory Architecture: Modern solvers such as Glucose 3.0+ and CaDiCaL divide learned clauses into three explicit tiers: Core ($ –

\text{LBD} , $permanentlykept$), Mid($ or 6, purged periodically based on activity), and Local (purged aggressively via FIFO or size). Integrating our Adaptive-Fixed composite scoring function into the Mid-tier management of a three-tier architecture could yield substantial improvements on multi-gigabyte industrial CNF benchmarks.

Online Entropy Sensing for Search-Phase Detection: Instead of relying solely on LBD moving averages, future restart controllers should monitor the Shannon entropy of variable assignments on the trail:

$$extTrail) = -\sum_i p_i \log_2 p_i$$

H(

A sudden collapse in assignment entropy indicates that the solver has entered a highlyconstrained,

deterministic resolution chain, during which restarts should be strictly locked out.

Bayesian Co-Optimization of Restart and Retention Hyper-Parameters: In this work, the weights of Adaptive-Fixed retention ($0.50, 0.40, 0.10$) and the V3 drawdown parameters ($D_{

ext{thresh}} = 0.

75, N_{

ext{persist}} = 3

0$) were selected via empirical calibration. Utilizing automated Bayesian hyper-parameter optimization (e.g., SMAC or Optuna) across diverse benchmark suites could identify optimal parameter sets tailored to specific problem classes.

Macro-Epoch Multi-Armed Bandits with Contextual Embeddings: To resolve the horizon mismatch identified in Chapter 8, future bandit architectures should operate at the macro-epoch scale (e.g., every 5,000 conflicts) and incorporate graph neural network (GNN) embeddings of the CNF bipartite graph to select the optimal restart engine based on instance structural topology.

References

Audemard, G., & Simon, L. (2009). *Predicting Learnt Clauses Quality in Modern SAT Solvers*. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009)*, pp. 399–404.

Audemard, G., & Simon, L. (2012). *Refining Restarts Strategies in SAT*. In *Principles and Practice of Constraint Programming (CP 2012)*, LNCS 7514, pp. 118–126. Springer.

Audemard, G., & Simon, L. (2018). *On the Glucose SAT Solver*. *International Journal on Artificial Intelligence Tools*, 27(1), 1840001.

Bayardo, R. J., & Schrag, R. C. (1997). *Using CSP Look-Back Techniques to Solve Real-World SAT Instances*. In *Proceedings of the 14th National Conference on Artificial Intelligence (AAAI 1997)*, pp. 203–208.

Biere, A., Heule, M., van Maaren, H., & Walsh, T. (Eds.) (2009). *Handbook of Satisfiability*. IOS Press.

Cook, S. A. (1971). *The Complexity of Theorem-Proving Procedures*. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC 1971)*, pp. 151–158.

Davis, M., Logemann, G., & Loveland, D. (1962). *A Machine Program for Theorem-Proving*. *Communications of the ACM*, 5(7), 394–397.

Eén, N., & Sörensson, N. (2003). *An Extensible SAT-solver [MiniSat]*. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, pp. 502–518. Springer.

Garivier, A., & Moulines, E. (2008). *On Upper-Confidence Bound Policies for Non-Stationary Bandit Problems*. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT 2008)*, pp. 174–188.

Gomes, C. P., Selman, B., Crato, N., & Kautz, H. (2000). *Heavy-Tailed Phenomena in Satisfiability and Constraint Satisfaction Problems*. *Journal of Automated Reasoning*, 24(1–2), 67–100.

Koriche, F., Lecoutre, C., Paparrizou, A., & Wattez, H. (2022). *Best Heuristic Identification for Constraint Satisfaction*. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI 2022)*, pp. 1859–1865.

Kurin, V., Godil, S., Whiteson, S., & Catanzaro, B. (2020). *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?* In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 9608–9621.

Li, C., Liu, C., Chung, J., Lu, Z., Jha, P., & Ganesh, V. (2024). *A Reinforcement Learning based Reset Policy for CDCL SAT Solvers*. *arXiv preprint arXiv:2404.03753*.

Liang, J., Cherif, S., & Li, C.-M. (2026). *Not All Restarts Are Equal: MAB-Learning at the Right Time Scale for SAT*. In *32nd International Conference on Principles and Practice of Constraint Programming (CP 2026)*, Leibniz International Proceedings in Informatics (LIPIcs), Volume 379, pp. 39:1–39:18. DOI: 10.4230/LIPIcs.CP.2026.39.

Liang, J. H., Ganesh, V., Poupart, P., & Czarnecki, K. (2016). *Learning Rate Based Branching Heuristic for SAT Solvers*. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, LNCS 9710, pp. 123–140. Springer.

Luby, M., Sinclair, A., & Zuckerman, D. (1993). *Optimal Speedup of Las Vegas Algorithms*. *Information Processing Letters*, 47(4), 173–180.

Marques-Silva, J. P., & Sakallah, K. A. (1996). *GRASP—A New Search Algorithm for Satisfiability*. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD 1996)*, pp. 220–227.

Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., & Malik, S. (2001). *Chaff: Accelerating SAT Design with Engineering*. In *Proceedings of the 38th Design Automation Conference (DAC 2001)*, pp. 530–535.

Tolpin, D., & Shimony, S. E. (2011). *Rational Deployment of CSP Heuristics*. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI 2011)*, pp. 680–686.

Wallace, R. J., Grimes, D., & Freuder, E. C. (2009). *Solving Dynamic Constraint Satisfaction Problems by Identifying Stable Features*. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009)*, pp. 621–627.

Appendices

Verification-based corrections to earlier report drafts

The final source-code and data audit identified formula and interpretation errors in several drafts. For this corrected submission version, the authoritative implementation values are:

V3 return: $r_t = \frac{(2/max(1, LBD))(1+\ln(1+\Delta D))}{1+\Delta P/25}$.

V3 control: $F_t = 0.92\,F_{t-1} + 0.08\,r_t$, $S_t = 0.99\,S_{t-1} + 0.01\,r_t$, $H_t = max(0.999\,H_{t-1}, F_t)$, and $D_t = (H_t - F_t)/(H_t + 0.05)$. A drawdown-based restart requires $D_t \geq 0.75$ for 30 consecutive conflicts, with applicable warmup/cooldown restrictions. A 2,500-conflict ceiling is separately enforced.

Adaptive-Fixed: $R(c) = 0.50\,S_{LBD} + 0.40\,S_{activity} + 0.10\,S_{size}$. Clause reduction follows first threshold 2000 and increment 300, subject to verified Glucose special postponement conditions. Glue and locked clauses are protected.

QRE: The implemented Sharpe-style signal uses 10-versus-50 step propagation-per-decision averages normalized by depth volatility. The VaR-inspired signal uses the maximum shock over a 12-conflict rolling window (nearest-rank 95th percentile), rather than $\mu + 1.645\,\sigma$.

Gated QRE: Requires simultaneous sustained alert signals; a 1,500-conflict cooldown and a maximum of two extra QRE restarts per 10,000-conflict block.

Portfolio-MAB: The implemented reward is conflict-normalized clause quality $clamp[(\Delta glue + 0.20\,\Delta nonglue)/\Delta c, 0, 1)$, not time-normalized reward.

Holdout interpretation: Glucose with Adaptive-Fixed has the lowest observed PAR-2; universal or statistically established superiority is not demonstrated. On hanoi5.cnf, V3 is faster than the no-heuristic control but slower than native Glucose.

Appendix A — Verified experimental results at a glance

Table A.1. Development suite: lower PAR-2 is better

Configuration	Solved	Timeouts	PAR-2 (s)	Restarts/run
V3 + Adaptive-Fixed	102/108	6	2.5381	1.1
Original Glucose	102/108	6	2.7463	73.2
Active QRE + Adaptive-Fixed	100/108	8	3.1960	91.7
Gated QRE + Adaptive-Fixed	98/108	10	3.4716	74.4
Glucose + Adaptive-Fixed	98/108	10	3.5892	66.7
Gated QRE + baseline retention	97/108	11	3.5915	64.3

Table A.2. Holdout suite: all configurations solved all 144 runs

Configuration	Solved	PAR-2 (s)	Conflicts
Glucose + Adaptive-Fixed	144/144	0.1107	709,587
Original Glucose	144/144	0.1208	778,158
V3 + baseline retention	144/144	0.1642	946,539
No-heuristic + Adaptive-Fixed	144/144	0.2417	1,129,974
V1 + baseline retention	144/144	0.2784	1,159,545
V3 + Adaptive-Fixed	144/144	0.3443	1,288,692
No-heuristic + baseline retention	144/144	0.4201	1,429,758
V1 + Adaptive-Fixed	144/144	0.4440	1,408,260

Table A.3. Hanoi-5 planning benchmark

Configuration	Mean time (s)	Conflicts	Genuine restarts
Glucose + Adaptive-Fixed	0.2061	15,407	57
Original Glucose	0.2887	17,213	58
V3 + baseline retention	3.0640	81,327	5
No-heuristic + baseline retention	15.2838	242,400	1

Appendix B — Source traceability and reproducibility

Implementation files: src/glucose_windows/core/Solver.cc, core/Solver.h, simp/Main.cc. **Development data:** experiments/results/qre_gated_full_development/master_results.csv. **Holdout data:** experiments/results/final_holdout/master_results.csv. **Verification:** PPT_VERIFICATION_REPORT.md (supplied audit, November 2026).

Interpretation caution: The supplied audit reports that 67% of holdout executions completed in 0.05 s or less. A raw paired Wilcoxon value around 0.0489 was reportedly recomputed, but corrected significance was not established. The older 0.184 p-value lacks a surviving automated source. Do not report an established statistically significant holdout advantage.

Verified implementation corrections: V3 return uses logarithmic backjump boost and linear propagation work; the drawdown threshold is 0.75 sustained for 30 conflicts, with a 50-conflict cooldown. QRE employs a nearest-rank 12-conflict shock statistic rather than a Gaussian VaR formula. Portfolio-MAB uses conflict-normalized reward rather than wall-clock-normalized reward.